Indian Institute of Technology Madras

CS6886 — Systems for Deep Learning

Accelerating SAM 3.1 Video Inference Through Temporal Computation Reuse

An Experimental Study of Eventful Transformers and Codec-Guided EP-ViT

Technical Report

Aravindh Sriram Kumar
CS26S020

Shaik Sadik
CS26S010

Department of Computer Science and Engineering
Indian Institute of Technology Madras

October 2026

Abstract

[TODO: Abstract.]

Keywords: video segmentation, vision transformers, temporal redundancy, token sparsity, GPU profiling.

Contents

Abstract ii

List of Figures v

List of Tables vi

List of Abbreviations vii

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Temporal Redundancy in Video Transformers . . . 2
- 1.3 Research Objectives . . . 3
- 1.4 Contributions . . . 5

2 Background and System Architecture 6
- 2.1 SAM 3.1 Architecture . . . 6
- 2.2 Vision Transformer Computation . . . 7
- 2.3 Eventful Transformers . . . 8
- 2.4 Video Codec Motion Information . . . 10

3 Problem Formulation 11
- 3.1 Temporal Computation Reuse . . . 11
- 3.2 Accuracy–Computation Trade-off . . . 12
- 3.3 GPU Execution Challenges . . . 13

4 Experimental Methodology 14
- 4.1 Hardware and Software . . . 14
- 4.2 Dataset . . . 14
- 4.3 Dense Baseline . . . 15
- 4.4 Evaluation Protocol . . . 16
- 4.5 Latency and Profiling Methodology . . . 16

5 Eventful Transformer Implementation 17
- 5.1 Integration into SAM 3.1 . . . 17
- 5.2 Token Gating and Feature Caching . . . 17
- 5.3 Layer-Selective Execution . . . 18
- 5.4 Implementation Optimizations . . . 18

6 Codec-Guided EP-ViT 18
- 6.1 Codec Motion Vectors and Residuals . . . 18
- 6.2 Mapping Codec Information to ViT Tokens . . . 19
- 6.3 Token Selection and Feature Reuse . . . 20
- 6.4 Gating and Implementation . . . 22

7 Evaluation Metrics 23
- 7.1 cgF1 and pHOTA . . . 24
- 7.2 Executed MACs . . . 25
- 7.3 Encoder and End-to-End Latency . . . 25

7.4 Speedup and Statistical Analysis 26

8 Experimental Results 27
8.1 Dense Baseline 27
8.2 Eventful Performance 28
8.3 Eventful Ablations 28
8.4 EP-ViT Performance 28

9 GPU Profiling and Bottleneck Analysis 33
9.1 Dense ViT Profiling 33
9.2 Sparse Execution Overhead 33
9.3 MAC Reduction vs. GPU Speedup 33
9.4 Kernel Optimization Opportunities 33

10 Discussion and Limitations 33

11 Conclusion and Future Work 33

References 34

A Hyperparameters and Configuration 36

B Additional Results 36

C Profiling Details 36

List of Figures

1.1 Twenty consecutive frames of a video clip, shown in raster order. Background, lighting and scene layout are nearly constant; the frame-to-frame differences are concentrated on the moving dancers. 1
1.2 Temporal redundancy of the two evaluation subsets at 24 fps. (a) MS-SSIM between frames t and $t+\Delta t$. (b) Cumulative distribution of optical-flow magnitude between consecutive frames, in pixels of the 1008×1008 model input; 14 px is one token. (c) Per-token cosine similarity of the ViT trunk (solid) and neck (dashed) outputs; dotted: trunk tokens of frames from different videos. (d) Share of trunk tokens whose cosine similarity with the previous frame is at least τ. Lines are means over the 10 videos of each subset; shaded bands in (a) and (c) are the inter-quartile range over videos. 3
1.3 Selective token recomputation in Eventful Transformers. Reproduced from Dutson et al. [1]. 4
1.4 Overview of EP-ViT: a spatiotemporal similarity analyser uses codec motion vectors and residuals to label tokens as static, matchable or unmatchable, and the ViT backbone recomputes only the tokens that require it, reading the remainder from an intermediate buffer. Reproduced from Qiu et al. [2]. 5
6.1 EP-ViT in the SAM 3.1 pipeline, with real data from `sav_012075` (frames 192–196). The Spatiotemporal Similarity Analyzer turns the H.264 residuals and motion vectors into token classes on the 72×72 grid and selects the top-k set S_t. The trunk computes those tokens and reuses the others from the Intermediate Buffer, whose rows are warped along the motion vectors. The Degradation-Aware Update Link forces tokens with accumulated error back into S_t. The neck, detector and tracker are unchanged. 18
6.2 From H.264 priors to the token plan of one P-frame. The figure is schematic: the frames are illustrative and the 12×12 grid stands for the real 72×72 grid. I-frames and the first frame of a session skip these steps and recompute all 5184 tokens. 20
8.1 Nsight Systems timelines on `normal` (`sav_012075`, “a mic stand”), at one time scale. In each panel, the top rows are the GPU kernels, and the rows below are the NVTX ranges of the frame, step and pipeline stages, where the neck range contains the trunk. The first full frame is the ninth captured frame in each panel, a slower frame than the median in all three methods. Screenshots of the Nsight Systems GUI, with process and memory rows removed. 30
8.2 The same view as Fig. 8.1 on `high_motion` (`sav_036663`, “black helmet”). 31
8.3 GPU timeline of one median frame, drawn from the Nsight Systems traces and coloured by pipeline stage. (a) `normal`: `sav_012075`, “a mic stand”. (b) `high_motion`: `sav_036663`, “black helmet”. Gaps are GPU idle time. Speedups are against dense B2d of the same pair. 31
8.4 Tokens computed and reused by EP-ViT. (a, c) One bar per frame at a 75 % budget for a `normal` and a `high_motion` video; black bars are I-frames, on which all tokens are computed. (b, d) Mean over the P-frames of all ten videos of the class at four budgets. Classes are the codec’s; tokens forced back by the update link appear under their codec class. 32

List of Tables

2.1 Analytic MAC count of one SAM 3.1 encoder forward pass, derived from the model configuration. Normalisation, softmax, activations and element-wise operations are excluded. 8
4.1 Hardware and software configuration. 14
4.2 Evaluation subsets of the SA-Co/VEval SA-V validation split. Model steps = native 24 fps frames fed to the pipeline over all pairs of the subset. 15
8.1 Dense SAM 3.1 at 24 fps on the common pairs. Accuracy ×100. Encoder = trunk time per frame; E2E = median per-frame model step; rest = E2E minus encoder; memory = median over pairs of the per-pair peak GPU allocation. Both presets execute 2699 GMAC per frame in the trunk. 27
8.2 End-to-end latency of EP-ViT on each video class against compiled dense SAM 3.1. The three methods of a class ran at the same time on separate GPUs over the same ten videos at 24 fps, so they saw the same host load. Encoder = trunk time per frame; E2E = median per-frame model step; FPS = 1000 / E2E. GMAC and pHOTA are from the full evaluation (70 `normal` and 119 `high_motion` pairs). . . 29
8.3 GPU time per frame by pipeline stage from Nsight Systems, in ms (median of 20 consecutive frames of one pair per class). Span = first kernel to last kernel of the step, including idle gaps between kernels, so it exceeds the sum of the stages. . . 29
8.4 Accuracy against end-to-end latency for every EP-ViT setting. α = token budget; Gated = operations that use the budget; GMAC = executed trunk GMAC per frame. pHOTA and cgF_1 are from the full evaluation (70 `normal` and 119 `high_motion` pairs). E2E is the controlled measurement of ten videos per class, and S_{e2e} is taken against the dense run of the same round; the dense row gives the mean of its four rounds. Bold: the setting we recommend. 32

List of Abbreviations

CUDA Compute Unified Device Architecture

EP-ViT Encoding-Prior Vision Transformer

FLOP Floating-Point Operation

FPS Frames Per Second

GOP Group of Pictures

GPU Graphics Processing Unit

HOTA Higher Order Tracking Accuracy

MAC Multiply–Accumulate Operation

MLP Multi-Layer Perceptron

MV Motion Vector

SAM Segment Anything Model

SA-Co Segment Anything with Concepts (benchmark)

SM Streaming Multiprocessor

ViT Vision Transformer

1 Introduction

Promptable concept segmentation in video asks a model to detect, segment and track every instance of an open-vocabulary concept across an entire clip. SAM 3.1 addresses this task with a single shared vision backbone that feeds both an image-level detector and a memory-based video tracker [3, 4]. This report studies whether the computation performed by that backbone can be reduced by exploiting the temporal redundancy inherent in video, and whether such reductions yield measurable acceleration on a GPU.

1.1 Motivation

The vision encoder of SAM 3.1 is a large ViTDet-style transformer [5] that converts each high-resolution frame into a dense grid of visual tokens, which are then consumed by the detector, the tracker and the mask decoder. Its cost is incurred once for every frame processed. The SAM 3.1 release introduced *Object Multiplex*, which groups tracked objects into fixed-capacity buckets and processes them jointly in order to remove redundant computation across objects [4]. This optimisation targets the per-object stages of the pipeline; the image encoder, by contrast, is executed per frame irrespective of how many objects are being tracked. In the released implementation each frame is encoded from scratch, and no intermediate encoder state is carried from one frame to the next. The encoder therefore constitutes a fixed per-frame cost that object-level optimisations do not address.

This per-frame recomputation is at odds with the structure of natural video. At typical capture rates, consecutive frames are dominated by the same background, illumination and scene layout, and differ mainly in regions undergoing object or camera motion (Fig. 1.1). A dense encoder nevertheless applies the full sequence of attention and MLP operations to every token of every frame, so a large share of the arithmetic it executes reproduces features that were already available from the preceding frame. Notably, SAM 3.1 already exploits temporal context downstream: its tracker maintains a memory of past frames and conditions its predictions on it. The encoder that feeds this memory, however, does not reuse its own computation over time.

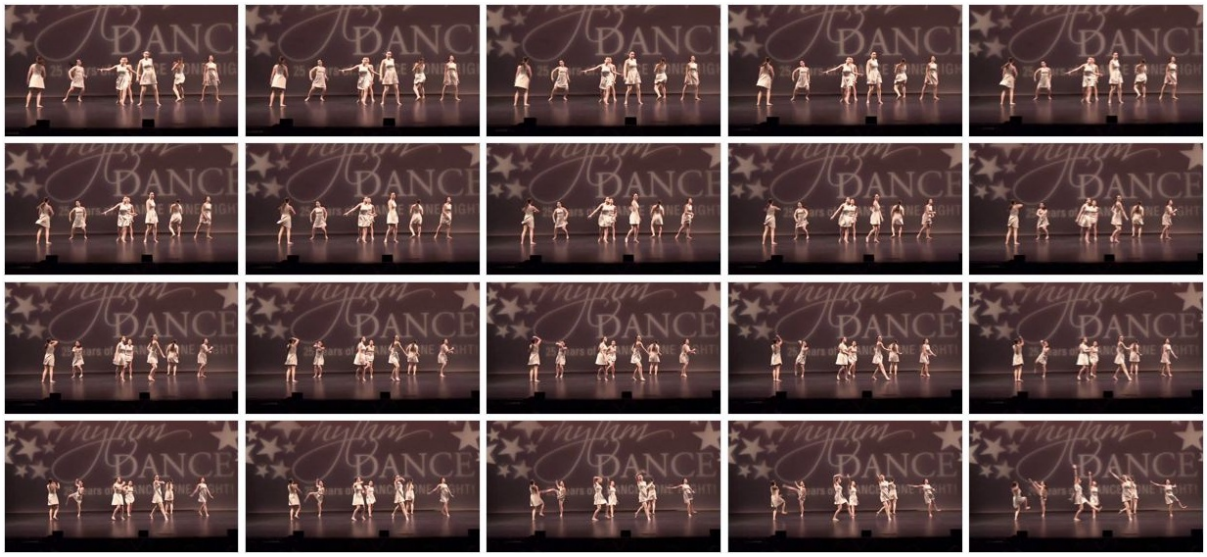


Figure 1.1: Twenty consecutive frames of a video clip, shown in raster order. Background, lighting and scene layout are nearly constant; the frame-to-frame differences are concentrated on the moving dancers.

We quantify this redundancy on the 20 SA-V validation videos used for evaluation, processed

at their native 24 fps (7,616 frames). They form two fixed subsets of 10 videos: the lowest-motion (*normal*) and the highest-motion (*high motion*) videos of the split. Because the subsets bracket the benchmark rather than sample it, we report them separately. We measure three complementary quantities (Fig. 1.2): structural persistence, as multi-scale SSIM [6] between frames t and $t + \Delta t$; motion sparsity, as dense Farnebäck optical flow [7] between consecutive frames, expressed in pixels of the 1008×1008 model input; and latent stability, as the per-token cosine similarity of the encoder outputs on the 72×72 token grid, computed through the predictor's own backbone and numerics.

On normal videos, consecutive frames are nearly identical. MS-SSIM is 0.98 and decays only to 0.79 after one second, 87% of pixels move by at most 1 px, and the encoder's output tokens have a mean cosine similarity of 0.86 (0.89 after the neck), with 69% of them above 0.9. High-motion videos behave very differently. MS-SSIM is 0.60 at a one-frame gap and falls to 0.30 within a quarter of a second, only 17% of pixels move by at most 1 px, the mean token similarity is 0.60, and only 15% of tokens exceed 0.9. Even there, however, 64% of token cells move by less than one token (14 px) per frame, and token similarity stays well above the 0.26 measured between tokens of unrelated videos. The redundancy is therefore substantial but uneven: it varies from token to token and from video to video, whereas the dense encoder spends the same computation on every frame. This argues for methods that decide *per token* what to recompute, rather than reusing or skipping whole frames.

1.2 Temporal Redundancy in Video Transformers

Redundancy in a video transformer has a spatial and a temporal component. Spatially, large homogeneous regions such as background produce tokens with similar content within a frame. Temporally, a token at a fixed position often changes little between adjacent frames, so its intermediate activations remain close to those computed for the previous frame. Eventful Transformers [1] turn this observation into a general mechanism for vision transformers: at selected operations, a *gate* compares each incoming token with a cached reference, a selection policy chooses the tokens whose change is significant, and only those tokens are recomputed, while a *buffer* supplies cached outputs for the remainder (Fig. 1.3). The computation performed per frame thus adapts to the amount of change in the scene, rather than being fixed by the input resolution.

Feature reuse is, however, an approximation. A reused token carries an activation that is correct for an earlier frame, and the discrepancy can grow with network depth: small changes at the input are mixed across tokens by attention and amplified through successive blocks, so the set of tokens that genuinely require recomputation may be considerably larger in the deep layers than at the input. In SAM 3.1 these approximations do not terminate at the encoder output. The encoder features drive detection, the association of instances across frames and the memory bank used by the tracker, so a reuse error can alter which instances are detected, how they are linked over time and what is written to memory for future frames. The relevant measure of fidelity is therefore downstream segmentation and tracking quality rather than the per-token reconstruction error of the encoder alone.

A second, independent difficulty concerns execution. Reducing the number of Multiply–Accumulate Operations (MACs) an algorithm requires does not by itself reduce the time a GPU needs to execute it. Dense GPU kernels are most efficient on large, regularly shaped tensors, whereas selective recomputation produces a different, data-dependent set of active tokens at every gated operation and every frame. Realising the arithmetic savings requires additional work

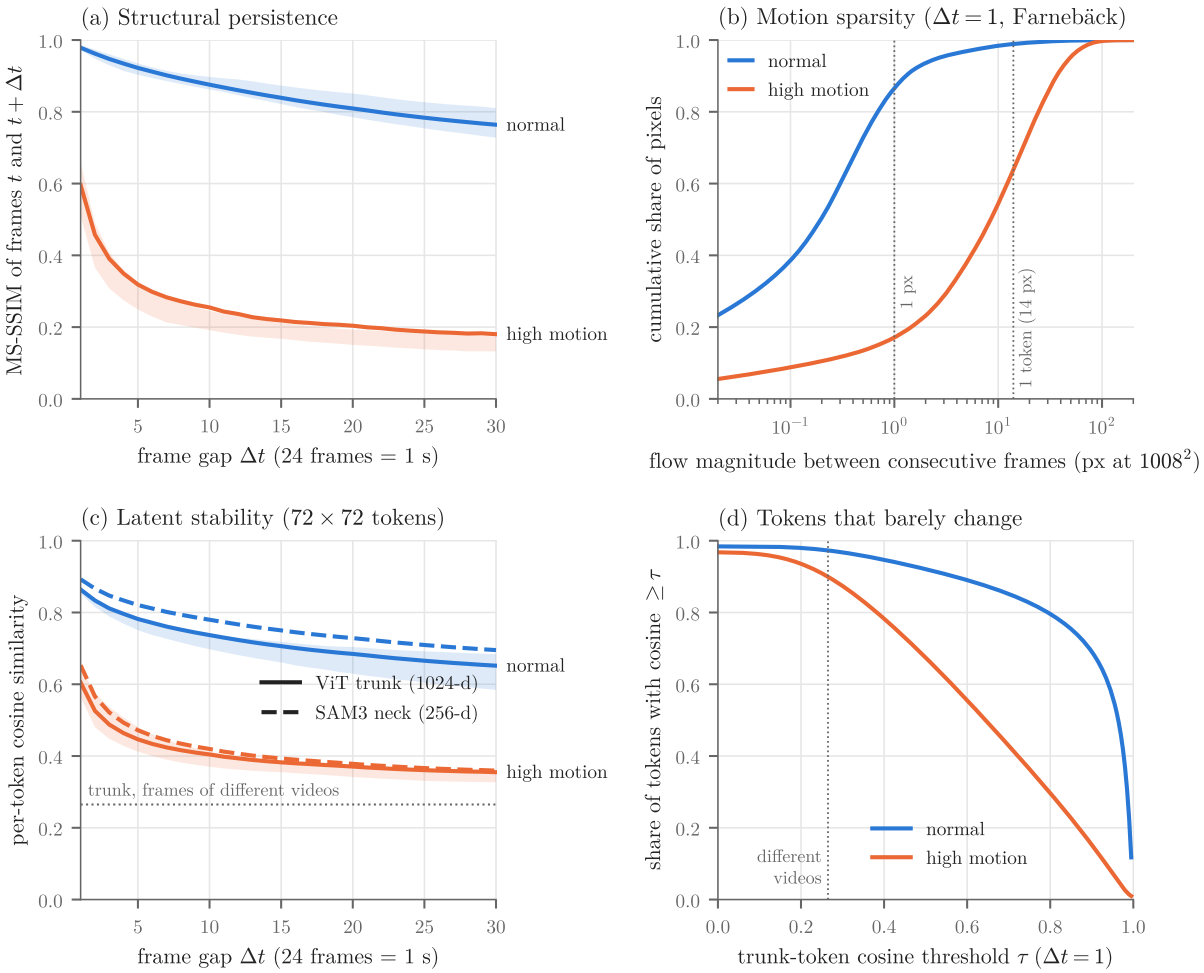
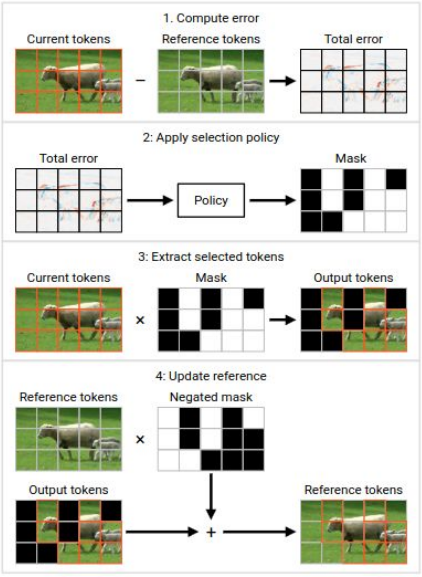


Figure 1.2: Temporal redundancy of the two evaluation subsets at 24 fps. (a) MS-SSIM between frames t and $t + \Delta t$. (b) Cumulative distribution of optical-flow magnitude between consecutive frames, in pixels of the 1008×1008 model input; 14 px is one token. (c) Per-token cosine similarity of the ViT trunk (solid) and neck (dashed) outputs; dotted: trunk tokens of frames from different videos. (d) Share of trunk tokens whose cosine similarity with the previous frame is at least τ. Lines are means over the 10 videos of each subset; shaded bands in (a) and (c) are the inter-quartile range over videos.

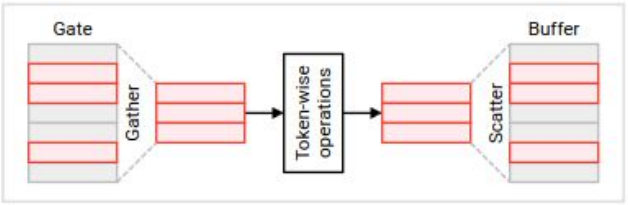
that a dense encoder never performs: computing change scores and selecting tokens (for example through a Top-K operation), gathering the selected tokens into a contiguous tensor, scattering results back into the cached buffers, and maintaining reference state for subsequent frames. Each of these steps moves data through GPU memory and typically adds kernel launches of its own. Moreover, when only a few tokens are active, the remaining matrix multiplications can become too small to occupy the GPU, so the computation shifts from being throughput-bound to being latency- and launch-bound. Consequently, the fraction of MACs eliminated is an upper bound on the attainable speedup rather than a prediction of it, and the two must be measured separately.

1.3 Research Objectives

This project investigates two approaches to temporal computation reuse in the SAM 3.1 vision encoder, which differ primarily in how they decide which tokens to recompute.



(a) Token gating: error against reference tokens, policy-based selection, extraction and reference update.



(b) Selected tokens are gathered into a compact tensor, processed by the token-wise operation and scattered back into a buffer of cached outputs.

Figure 1.3: Selective token recomputation in Eventful Transformers. Reproduced from Dutson et al. [1].

A. Eventful Transformers. The first approach adapts Eventful Transformers [1] to the SAM 3.1 encoder. Token selection is driven by feature change: each gate measures how far a token's current input has drifted from the input at which it was last recomputed, and a selection policy determines which tokens are updated. The decision is made from the network's own activations, and therefore requires the change scores to be computed at inference time.

B. Codec-guided EP-ViT. The second approach follows EP-ViT [2], which derives the token selection from information already produced by a video codec. Block-based codecs such as H.264/AVC [8] and HEVC [9] encode most frames predictively, describing each block by a motion vector that points to a matching region in a reference frame and a residual that captures what the prediction fails to explain. EP-ViT maps these motion vectors and residuals onto the transformer's token grid, classifies tokens as static, matchable or unmatchable, and recomputes only the tokens whose content cannot be recovered from cached features (Fig. 1.4). Because the decision is taken before the encoder runs, it does not depend on intermediate activations.

Both approaches are evaluated with respect to two central research questions:

RQ1 How much Vision Transformer (ViT) computation can be eliminated while preserving the downstream segmentation and tracking accuracy of SAM 3.1?

RQ2 To what extent do these computational savings translate into measurable GPU inference acceleration?

RQ1 concerns the algorithmic trade-off between executed computation and accuracy, and is answered in terms of executed MACs and downstream metrics. RQ2 concerns the systems question of whether that trade-off survives contact with real hardware, and is answered through measured latency and kernel-level profiling.

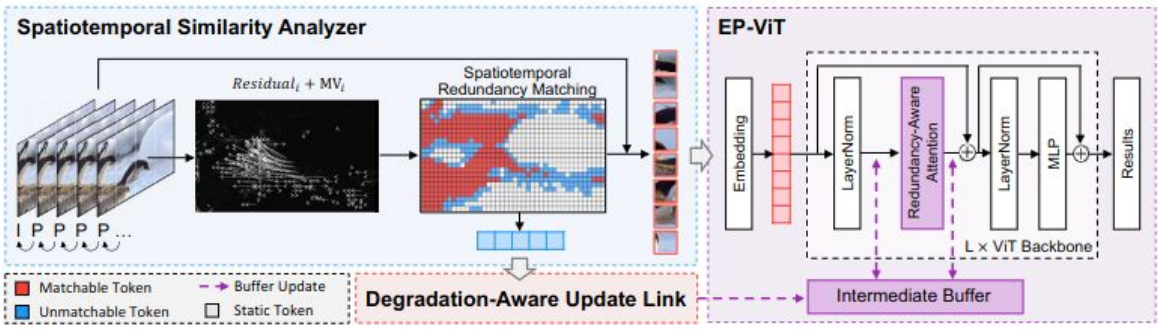


Figure 1.4: Overview of EP-ViT: a spatiotemporal similarity analyser uses codec motion vectors and residuals to label tokens as static, matchable or unmatchable, and the ViT backbone recomputes only the tokens that require it, reading the remainder from an intermediate buffer. Reproduced from Qiu et al. [2].

1.4 Contributions

The scope of this work is an experimental study of temporal reuse inside an otherwise unmodified SAM 3.1 video pipeline. The detector, tracker, memory and mask decoder are left intact, and only the execution of the vision encoder is changed, so that any change in downstream quality can be attributed to the reuse mechanism. Within this scope the report makes the following contributions:

- The integration of temporal computation reuse into the SAM 3.1 vision encoder, operating within the chunked video inference pipeline that feeds the detector and the tracker (Sections 5 and 6).
- An investigation of two token-selection strategies within this setting: feature-change-based gating following Eventful Transformers, and codec-guided selection following EP-ViT, for which codec information is obtained by re-encoding the evaluation frames (Section 6).
- Ablations over the token budget, over the operations that are gated, and over the encoder layers to which reuse is applied (Section 8).
- An accuracy evaluation using the cgF$_1$ and pHOTA metrics, which capture segmentation and tracking quality at the level of the full pipeline (Sections 7 and 8).
- An efficiency analysis that reports executed MACs, encoder latency and end-to-end latency side by side, relative to a dense baseline (Sections 7 and 8).
- GPU profiling of the dense and sparse encoders to identify the bottlenecks of sparse execution and to explain the gap between MAC reduction and wall-clock speedup (Section 9).

The remainder of the report is organised as follows. Section 2 reviews the SAM 3.1 architecture, Eventful Transformers and codec motion information. Section 3 formalises temporal computation reuse and the associated trade-offs. Sections 4 and 7 describe the experimental methodology and evaluation metrics. Sections 5 and 6 present the two implementations, Section 8 reports the experimental results, and Section 9 analyses GPU behaviour. Sections 10 and 11 discuss limitations and conclude.

2 Background and System Architecture

This section describes the SAM 3.1 pipeline, formalises the computation of its vision encoder, and introduces the two sources of reuse decisions studied in this report: feature change (Eventful Transformers) and video codec motion information (EP-ViT). Our adaptations of both are deferred to Sections 5 and 6. Throughout, the *encoder* (or *ViT trunk*) denotes the 32-block vision transformer; the *backbone* denotes the encoder together with its feature-pyramid neck; and the *pipeline* denotes the complete SAM 3.1 video inference system. Temporal reuse is applied only to the encoder.

2.1 SAM 3.1 Architecture

SAM [10] introduced promptable image segmentation, and SAM 2 [11] extended it to video with a streaming memory. SAM 3 [3] generalises prompts to *concepts* (noun phrases or image exemplars) and returns masks and persistent identities for every matching instance, using an image-level detector and a memory-based tracker that share one vision backbone. SAM 3.1 [4] keeps this architecture and adds Object Multiplex, which processes tracked objects jointly in fixed-capacity buckets. The description below follows the released SAM 3.1 multiplex video predictor, whose configuration we inspected directly.

Frame encoding. Each frame is resized to 1008×1008 pixels (without preserving the aspect ratio) and normalised. The encoder, a ViTDet-style transformer [5] derived from the Perception Encoder [3, 12], produces a 72×72 grid of 1024-dimensional tokens. A tri-head neck converts the trunk output into three multi-scale feature pyramids (288^2, 144^2 and 72^2, 256 channels), one each for detection, interactive prompting and tracker propagation. The trunk runs once per frame, and its output is shared by all three heads.

Prompt conditioning and detection. The text prompt is encoded once and cached. On every frame, the detector fuses the image features with the prompt embedding, decodes object queries with a DETR-style decoder [13], and predicts boxes, scores and masks. A presence token estimates whether the concept occurs in the frame at all [3]. The detector is responsible for discovering new instances.

Memory, mask prediction and temporal propagation. The tracker propagates existing masklets. A memory-attention transformer conditions the current frame's features on a memory bank of encoded features and masks from past frames (`num_maskmem` = 7 in the released configuration) and on object pointer tokens. A SAM-style mask decoder then predicts each object's mask, and a memory encoder writes the result back to the memory bank. These steps run per bucket of up to 16 objects. Detector and tracker outputs are then reconciled: unmatched detections start new masklets, and existing masklets are confirmed or suppressed.

Repeated computation. During propagation, the encoder and detector process chunks of consecutive frames, and the tracker reads the cached features of each frame without invoking the encoder. The chunk size does not change the encoder's per-frame work. The work divides into three classes. Text encoding runs once per prompt. Tracker work scales with the number of object buckets, which is what Object Multiplex optimises. The backbone and detector run once per frame, regardless of how many objects are tracked. The tracker reuses information over

time through its memory, but the trunk recomputes every token of every frame from scratch and carries no state between frames. As the largest component, and one whose cost is paid on every frame, the encoder is the target of this study.

2.2 Vision Transformer Computation

A vision transformer [14] splits an image $I \in \mathbb{R}^{3\times H\times W}$ into $N = HW/P^2$ patches of size $P \times P$ and embeds each one as a D-dimensional token,

$$\boldsymbol{x}_i^{(0)} = E\ \mathrm{vec}(I_i) + \boldsymbol{p}_i, \qquad i = 1, \ldots, N, \tag{2.1}$$

where I_i is the i-th patch, $E \in \mathbb{R}^{D\times 3P^2}$ is the patch projection and $\boldsymbol{p}_i \in \mathbb{R}^D$ is a positional embedding. The rows $\boldsymbol{x}_i^{(0)}$ form $X^{(0)} \in \mathbb{R}^{N\times D}$. Each of the L pre-normalisation blocks [15] computes

$$Y = X + \mathrm{MHSA}\big(\mathrm{LN}(X)\big), \qquad X' = Y + \mathrm{MLP}\big(\mathrm{LN}(Y)\big), \tag{2.2}$$

where X and X' are the block input and output and LN is per-token layer normalisation [16]. With $Z = \mathrm{LN}(X)$, multi-head self-attention with H_a heads of width $d_h = D/H_a$ is

$$[\,Q\ K\ V\,] = Z\,W_{QKV} + \mathbf{1}\boldsymbol{b}_{QKV}^{\top}, \qquad O_h = \underbrace{\mathrm{softmax}\Big(\tfrac{\tilde{Q}_h\tilde{K}_h^{\top}}{\sqrt{d_h}}\Big)}_{A_h} V_h, \tag{2.3}$$

$$\mathrm{MHSA}(Z) = [\,O_1, \ldots, O_{H_a}\,]\,W_O + \mathbf{1}\boldsymbol{b}_O^{\top}, \tag{2.4}$$

where $W_{QKV} \in \mathbb{R}^{D\times 3D}$ is the fused query/key/value projection, $Q_h, K_h, V_h \in \mathbb{R}^{N\times d_h}$ are the per-head column blocks, $\tilde{Q}_h, \tilde{K}_h$ are the queries and keys after the positional rotation described below, A_h is the row-wise softmax attention matrix, $W_O \in \mathbb{R}^{D\times D}$ is the output projection, $\boldsymbol{b}$ are bias vectors, and $\mathbf{1} \in \mathbb{R}^N$ is the all-ones vector. The MLP acts on each token $\boldsymbol{z}$ independently,

$$\mathrm{MLP}(\boldsymbol{z}) = W_2\,\sigma(W_1\boldsymbol{z} + \boldsymbol{b}_1) + \boldsymbol{b}_2, \qquad W_1 \in \mathbb{R}^{D_h\times D},\ W_2 \in \mathbb{R}^{D\times D_h}, \tag{2.5}$$

with hidden width D_h and GELU nonlinearity σ [17].

The SAM 3.1 encoder. The configuration is $H = W = 1008$, $P = 14$ ($N = 5184$), $D = 1024$, $L = 32$, $H_a = 16$, $d_h = 64$ and $D_h = 4736$ (MLP ratio 4.625). Positions are encoded in two ways: by an absolute embedding table tiled over the 72×72 grid (Eq. (2.1)), and by 2D axial rotary embeddings (RoPE) [18] applied to the queries and keys of every block. No class token is used at inference. Following ViTDet [5], 28 blocks use *windowed* attention: the grid is split into 3×3 windows of $N_w = 24 \times 24 = 576$ tokens, and Eq. (2.3) is evaluated within each window. The other four blocks, with zero-based indices 7, 15, 23 and 31, use *global* attention over all N tokens. Windowed blocks exchange no information across window boundaries, so information crosses windows only in the four global blocks.

Computational cost. Counting multiply–accumulate operations (MACs) and ignoring normalisation, softmax, activations and additions, each token in each block costs $3D^2$ MACs for the QKV projection, D^2 for the output projection and $2DD_h$ for the MLP. These costs are independent of all other tokens. The two attention products cost $2N_aD$ MACs per query, summed over heads, where the attention span N_a is N_w in windowed blocks and N in global blocks. A block's attention therefore costs $2NN_aD$, which is quadratic in N for global blocks.

Table 2.1: Analytic MAC count of one SAM 3.1 encoder forward pass, derived from the model configuration. Normalisation, softmax, activations and element-wise operations are excluded.

Operation	GMAC / block	Blocks	GMAC total
QKV projection ($3ND^2$)	16.31	32	521.8
Output projection (ND^2)	5.44	32	173.9
MLP ($2NDD_h$)	50.28	32	1609.0
Windowed attention ($2NN_wD$)	6.12	28	171.2
Global attention ($2N^2D$)	55.04	4	220.2
Patch embedding ($3P^2ND$)	–	–	3.1
Total			**2699.3**

Table 2.1 evaluates these expressions for the SAM 3.1 configuration: the token-wise linear layers dominate the arithmetic.

Token-wise operations and selective computation. An operation is *token-wise* if the output for token i depends only on the input for token i. Patch embedding, layer normalisation, the QKV and output projections, the MLP and the residual additions are all token-wise. If a token's input is unchanged, its output under such an operation f is unchanged, so a cached output can be reused exactly. Let $\mathcal{S}_t \subseteq \{1, \dots, N\}$, $|\mathcal{S}_t| = M$, be the set of *active* tokens recomputed at frame t, and let $P_t \in \{0,1\}^{M \times N}$ be its row-selection matrix. Then

$$\text{dense:} \quad Y_t = f(X_t), \qquad \text{cost } Nc_f, \tag{2.6}$$

$$\text{selective:} \quad Y_t = (I_N - P_t^\top P_t)\, Y_{t-1} + P_t^\top f(P_t X_t), \qquad \text{cost } Mc_f, \tag{2.7}$$

where X_t and Y_t are the input and output of f at frame t, c_f is the per-token MAC cost of f, and Y_{t-1} is the cached output. $P_t X_t$ *gathers* the active rows, and multiplication by $P_t^\top$ *scatters* the results back into the cache. The two forms are equal when $\mathcal{S}_t$ contains every token whose input changed; otherwise the cached rows are approximations.

Why fewer active tokens do not reduce every cost. Equation (2.7) scales the MACs of f by M/N, but not the cost of the whole block. Attention is not token-wise: row i of O_h depends on the keys and values of its entire span, so an unchanged query still has a changed output if any of them changes, and all keys and values must remain available. Selection itself examines all N tokens and accesses memory at data-dependent indices. Finally, MACs omit memory-bound operations (normalisation, residual additions, layout changes) and say nothing about how efficiently small, irregular matrix products use the GPU. We therefore treat MAC reduction as a theoretical quantity and measure time separately (Section 7).

2.3 Eventful Transformers

Eventful Transformers [1] realise Eq. (2.7) with two stateful modules, a *gate* and a *buffer*, and choose $\mathcal{S}_t$ from the observed change of each token. An existing transformer can often be converted without retraining [1].

Caching and change-based selection. A gate receives the current input $C_t \in \mathbb{R}^{N \times D}$ and keeps a reference $U \in \mathbb{R}^{N \times D}$ holding, for each token, the value at which it was last recomputed.

It computes the error and per-token score

$$E_t = C_t - U_{t-1}, \qquad s_{t,i} = \|E_{t,i}\|_2 , \tag{2.8}$$

where $E_{t,i}$ is row i of E_t. (The original defines the error with the opposite sign; the score is the same.) The *top-r* policy selects the r highest-scoring tokens, $\mathcal{S}_t = \mathrm{TopK}(\{s_{t,i}\}_{i=1}^{N}, r)$, so the token budget r fixes the cost in advance. The *threshold* policy selects $\mathcal{S}_t = \{i : s_{t,i} > h\}$. This adapts the cost to the scene, but the cost is not fixed, and a good value of h depends on token-norm statistics that vary across layers [1]. The gate passes the gathered rows $\tilde{C}_t = P_t C_t$ downstream and updates the reference only for active tokens, $U_t = (I_N - P_t^\top P_t)U_{t-1} + P_t^\top \tilde{C}_t$. The paired buffer scatters $f(\tilde{C}_t)$ into its stored outputs, which restores a full $N \times D$ tensor as in Eq. (2.7). All tokens are selected on the first frame. Because inactive references are not updated, an inactive token's error accumulates until the token is selected.

Gating within a block. Gate–buffer pairs can enclose any token-wise operation. In the Eventful block [1], gates control recomputation of the QKV projection, the output projection and the MLP. The original block also makes attention sparse. In $\tilde{Q}_h \tilde{K}_h^\top$, only the rows of updated queries and the columns of updated keys are recomputed. The attention–value product is updated through the identity

$$A_n \boldsymbol{v}_n = A_o \boldsymbol{v}_o + A_n \boldsymbol{v}_\Delta + A_\Delta \boldsymbol{v}_n - A_\Delta \boldsymbol{v}_\Delta, \tag{2.9}$$

where the subscripts o, n and Δ denote old values, new values and their (sparse) differences. This form requires the attention matrix itself to be cached.

Approximation error. An inactive token contributes $f(U_{t-1,i})$ in place of $f(C_{t,i})$, and the error propagates in two directions. Across blocks, it perturbs the token's residual stream and therefore the inputs to all later gates; stale keys and values also perturb the attention outputs of *other* tokens. Across frames, it persists in the caches until the token is reselected or the caches are reset by a dense recomputation. The threshold policy bounds each gate's input error by h. The top-r policy bounds cost instead, and can leave large changes unselected when more than r tokens change.

Selection and irregular-access overhead. Beyond the gather and scatter, each gate reads C_t and U_{t-1} for all N tokens and keeps $N \times D$ state per video. Top-r requires a partial sort, and a threshold policy yields an active set whose size is known only at run time. RoPE must be evaluated at the active indices, and active tokens are spread unevenly across windows, which complicates batching [1]. The original implementation used standard PyTorch operators; its authors present their runtimes as a proof of concept and name a fused CUDA kernel for the gating logic as a possible improvement [1].

Original mechanism versus this study. The original evaluation uses standalone video backbones, ViTDet for video object detection and ViViT for action recognition, whose outputs feed a task head directly [1]. In SAM 3.1, the encoder output feeds a detector, a tracker and a temporal memory, so reuse errors can affect detection, association and future memory reads. Our implementation follows the token-wise form of the Eventful block, with gates on the QKV, output projection and MLP inputs, and adapts it to the windowed/global structure and rotary

embeddings of the SAM 3.1 encoder. Section 5 describes it, including where it deviates from the original.

2.4 Video Codec Motion Information

Inter-frame prediction. H.264/AVC [8] and HEVC [9] are block-based hybrid codecs. I-frames are coded independently; P- and B-frames are predicted from previously decoded *reference frames*. Frames are partitioned into blocks: 16×16 macroblocks (with partitions down to 4×4) in H.264, and quadtree-split coding tree units of up to 64×64 in HEVC. For an inter-coded block $\mathcal{B}$ of frame t, the encoder signals a reference index $r_{\mathcal{B}}$ and a *motion vector* $\boldsymbol{v}_{\mathcal{B}} \in \mathbb{R}^2$, and codes the *residual* of the motion-compensated prediction:

$$\hat{I}_t(\boldsymbol{p}) = \tilde{I}_{r_{\mathcal{B}}}(\boldsymbol{p} + \boldsymbol{v}_{\mathcal{B}}), \qquad R_t(\boldsymbol{p}) = I_t(\boldsymbol{p}) - \hat{I}_t(\boldsymbol{p}), \qquad \boldsymbol{p} \in \mathcal{B}, \tag{2.10}$$

where $\boldsymbol{p}$ is a pixel position, I_t is the source frame and $\tilde{I}_r$ is a decoded reference frame. Motion vectors have quarter-sample precision. The residual is transformed, quantised and entropy-coded, and the decoder reconstructs $\tilde{I}_t = \hat{I}_t + \tilde{R}_t$ from the dequantised residual $\tilde{R}_t$. The encoder selects each block's partition, reference and motion vector by rate–distortion optimisation, $\arg\min (D_{\mathcal{B}} + \lambda R_{\mathcal{B}})$, where $D_{\mathcal{B}}$ is the block's distortion, $R_{\mathcal{B}}$ is its bit cost and λ is a Lagrange multiplier [19].

Motion vectors versus optical flow. Optical flow estimates per-pixel apparent motion, typically from brightness constancy, $I(\boldsymbol{p} + \boldsymbol{u}(\boldsymbol{p}), t+1) = I(\boldsymbol{p}, t)$, together with a smoothness prior [20]. Motion vectors differ in four ways. They are per block rather than per pixel. They are chosen to minimise coding cost rather than to recover physical motion, so a vector may point to any similar block. They may reference frames other than the preceding one. They are absent for intra-coded blocks. Their appeal is that the decoder provides them at negligible cost, as earlier compressed-domain recognition work has exploited [21].

Codec blocks versus ViT tokens. Codec blocks are defined at native resolution and vary in size, whereas the SAM 3.1 encoder resizes every frame to 1008×1008 and uses fixed 14×14 patches. A token thus covers a native region of $(W/72) \times (H/72)$ pixels, which is generally neither square nor aligned with block boundaries, and may overlap several blocks with different motion vectors and residuals. Token-level use therefore requires rescaling the motion vectors and aggregating block signals within each token's footprint, and both steps lose information.

Codec signals as change indicators, and their limits. Motion vectors and residuals suggest a classification of regions: zero motion with a small residual indicates approximately static content; non-zero motion with a small residual indicates displaced content; and a large residual or intra coding indicates content that no reference predicts well, such as occlusion, deformation or an illumination change. These are, however, *pixel-domain* signals, and they do not directly measure change in the encoder's features. First, deep features have large receptive fields (a full window, or the whole frame after a global block), so a token with static pixels can acquire new features when content elsewhere changes. Second, the encoder is not translation-equivariant at the token level: displaced content arrives at a new absolute position, with a different positional embedding, rotary phase and window context, and its displacement is generally not a multiple of the patch size. Third, motion vectors are optimised for compression, and residuals measure

quantised pixel error rather than semantic change. Codec information is therefore an inexpensive *prior* on where features are likely to have changed, not a measurement of feature change.

Codec-guided token recomputation. Eventful Transformers measure change in the network's own activations, at a cost paid at every gate on every frame. A codec-guided approach instead derives the recomputation decision, before the encoder runs, from signals the decoder already provides. EP-ViT [2] realises this idea. It maps motion vectors and residuals onto the token grid, classifies tokens as static, matchable or unmatchable, and recomputes only those whose content cannot be recovered from cached features (Fig. 1.4). Given the limitations above, whether such priors select the tokens that matter for the SAM 3.1 detector and tracker is an empirical question. Our adaptation is described in Section 6.

3 Problem Formulation

This section states the problem both methods solve. Section 3.1 defines temporal computation reuse in the SAM 3.1 encoder as a per-frame choice of tokens to recompute, Section 3.2 defines the trade-off between executed computation and downstream accuracy along which the methods are compared (RQ1), and Section 3.3 explains why a reduction in computation need not become a reduction in GPU time (RQ2).

3.1 Temporal Computation Reuse

Setting. A video arrives as a sequence of frames $x_1, x_2, \ldots, x_T$, and the pipeline processes them online, one frame per step and in order. Dense inference computes the encoder features $F_t = \mathcal{E}(x_t)$ of every frame from scratch, where $\mathcal{E}$ is the 32-block encoder of Section 2.2. A reuse method instead computes approximate features $\hat{F}_t$ from the current frame and a state $\mathcal{C}_{t-1}$ of cached intermediate results,

$$(\hat{F}_t, \mathcal{C}_t) = \hat{\mathcal{E}}(x_t, \mathcal{C}_{t-1}; \pi), \tag{3.1}$$

and passes $\hat{F}_t$ to the unmodified neck, detector and tracker. The *policy* π decides, for each frame t and each gated operation f of block l, the set $\mathcal{S}_t^{(l,f)} \subseteq \{1, \ldots, N\}$ of active tokens that are recomputed. All other tokens take their value from the cache.

Reuse with motion. Equation (2.7) reuses a token's cached value at the same grid position. When content moves, the best cached value for token i may belong to another token. We therefore generalise the selective update to

$$Y_t = (I_N - P_t^\top P_t)\, \Pi_t\, Y_{t-1} + P_t^\top f(P_t X_t), \tag{3.2}$$

where $\Pi_t \in \{0,1\}^{N \times N}$ copies each inactive token's cached row from its *source* token. Eventful Transformers use $\Pi_t = I_N$: a token is either recomputed or keeps its own cached value. EP-ViT sets row i of Π_t from the codec motion vector of token i (Section 6.3).

Where the decision comes from. The two methods differ in the information π uses:

- *Activation-driven* (Eventful): $\mathcal{S}_t^{(l,f)}$ is chosen inside the encoder, at every gate, from the change between the operation's current input and the input at which each token was last recomputed. The decision is specific to each operation and block, and its cost is paid on every frame.

- *Prior-driven* (EP-ViT): one set $\mathcal{S}_t$ and one copy map Π_t are chosen before the encoder runs, from the motion vectors and residuals of an H.264 bitstream. The decision is shared by all blocks and costs no Graphics Processing Unit (GPU) work.

Refreshes. Both methods occasionally run a *flush*, a dense forward pass that rebuilds the whole cache: on the first frame of every session, when frames arrive out of order, and, for EP-ViT, on every codec I-frame. A flush bounds how long approximation errors can accumulate in the cache.

Constraints. We keep four constraints throughout. The method is *training-free*: SAM 3.1's weights are used unchanged. Only the *encoder* is modified; the neck, detector, tracker and memory are SAM 3.1's own, so any change in accuracy is caused by the reuse. Inference is *online* (batch 1, causal), as in a streaming deployment. Finally, the number of recomputed tokens per gated operation is a *fixed budget* k, i.e. $|\mathcal{S}_t| = k$ on every frame that is not a flush; which tokens are recomputed is chosen per frame. The reason for this choice is given in Section 3.3.

3.2 Accuracy–Computation Trade-off

Executed computation. For a policy π we measure the MACs the encoder actually executes, averaged over the frames it encodes,

$$C(\pi) = \frac{1}{T}\sum_{t=1}^{T}\Big(\sum_{l=1}^{L}\sum_{f}|\mathcal{S}_t^{(l,f)}|\,c_f + C_t^{\text{attn}} + C^{\text{embed}}\Big), \tag{3.3}$$

where c_f is the per-token cost of operation f (Table 2.1), C_t^{attn} is the attention actually executed and C^{embed} is the patch embedding. On a flush every set contains all N tokens, and C equals the dense cost $C_{\text{dense}} = 2699\,\text{GMAC}$. We report the reduction $1 - C(\pi)/C_{\text{dense}}$. The counting rules are given in Section 7.2.

Downstream accuracy. Accuracy is measured on the output of the whole pipeline, not on the encoder features: $A(\pi)$ is the pHOTA or cgF$_1$ score of the predicted masklets (Section 7.1). A feature error does not map to a fixed accuracy loss. The detector may still find the same objects, while the tracker writes reused features into its memory bank and carries them into later frames. An error that would be harmless on one frame can therefore grow, or be corrected, over time. Only the pipeline-level metric captures this.

The trade-off. Each policy is a point $(C(\pi), A(\pi))$, and a method is described by the set of points it reaches as its parameters vary. We do not optimise a single objective. Instead we sweep the parameters that set the compute (the budget k, the gated operations and the gated blocks) and compare the methods by the accuracy they keep at equal executed computation, relative to the dense pipeline $(C_{\text{dense}}, A_{\text{dense}})$. A method is preferred at a given compute if it has higher accuracy there. This answers RQ1.

From computation to time. RQ2 asks how much of $1 - C(\pi)/C_{\text{dense}}$ becomes measured time. Reuse only shortens the encoder, which takes a fraction ϕ of the pipeline's per-frame time.

If the encoder becomes s times faster, the end-to-end speedup is bounded by Amdahl's law,

$$S_{\text{e2e}} = \frac{1}{(1-\phi) + \phi/s} \;\leq\; \frac{1}{1-\phi}. \tag{3.4}$$

For the dense SAM 3.1 pipeline we measured $\phi \approx 0.53$ of the GPU time per step (Section 9), so even an infinitely fast encoder would make the pipeline at most about 2.1× faster, and a 1.3× faster encoder gives at most 1.14×. We therefore report encoder and end-to-end speedups separately.

3.3 GPU Execution Challenges

Equation (3.3) counts arithmetic only. On a GPU, a sparse encoder also pays for work that a dense encoder does not do, and does its remaining arithmetic less efficiently. We expect the following effects to limit the speedup; Section 9 measures them.

Smaller matrix products. The linear layers of a dense block are matrix products with $N = 5184$ rows, large enough to keep all Streaming Multiprocessors (SMs) busy. With reuse they have k rows. The work shrinks with k, but the time shrinks less, because smaller products use the hardware less efficiently and launch overhead is unchanged.

Gather, scatter and selection. The active rows must be gathered before each gated operation and scattered back into the cache afterwards (Eq. (2.7)). These copies move 1024-wide rows at data-dependent indices and perform no arithmetic, so they are limited by memory bandwidth. Activation-driven selection adds a pass over all N tokens to score their change and a top-k selection, in every gated operation of every block. Prior-driven selection moves this work to the host, but must upload index tensors and, for warped tokens, copy cached rows in every block.

Attention does not shrink with the token set. Attention is not token-wise: every recomputed query still attends to the keys and values of its whole window or of the whole frame, so these must be kept for all N tokens and, with RoPE, rotated for all positions. In the 28 windowed blocks the active tokens are spread unevenly over the nine windows, and batching the windows requires padding every window to the largest one.

Data-dependent shapes and synchronisation. If the number of active tokens depended on the content, for example on a change threshold, the host would have to wait for the GPU to report the count before it could launch the next operation. This synchronisation stalls the launch pipeline, and the varying tensor shapes prevent `torch.compile` and CUDA graphs from using static kernels. A fixed budget k avoids both: shapes are known in advance and selection stays on the device (Eventful) or on the host (EP-ViT) without a round trip.

Launch overhead at batch 1. Online inference runs one frame at a time, so the encoder launches many short kernels. Every gather, scatter, selection and cache copy adds kernels in each of the 32 blocks, and at batch 1 their launch cost is not hidden by large kernels. Compilation can fuse some of them, but not all.

Table 4.1: Hardware and software configuration.

Component	Specification
GPU	NVIDIA RTX A5000 (Ampere), 24 GiB GDDR6, 230 W power cap; one per run
CPU	Intel Xeon w5-3435X, 16 cores / 32 threads
Host memory	125 GiB
OS	Ubuntu 22.04.5 LTS, Linux 6.8
Driver / CUDA	NVIDIA driver 580.178.04; CUDA 12.8, cuDNN 9.10
Python / PyTorch	Python 3.12.14; PyTorch 2.10.0 (`torch.compile`, Inductor), Triton 3.6.0
Model	SAM 3.1 multiplex video predictor, official checkpoint
Codec	FFmpeg 4.4.2 with `libx264`; PyAV 19.0.2
Profilers	Nsight Systems 2026.3.1, Nsight Compute 2026.2.1, `torch.profiler`

Memory. The caches add memory proportional to $L \cdot N \cdot D$: about 0.65–0.95 GiB, depending on what is cached (Sections 5 and 6). At 24 fps SAM 3.1's tracker already keeps per-object state for every frame on the GPU, and the longest, most crowded videos exceed the 24 GiB of our GPU even with the dense encoder.

A strong dense baseline. Finally, the reference must itself be efficient. Free optimisations of the dense encoder, such as bf16 weights, fused RoPE and compilation, are applied to the baseline as well (Section 4.3), so reuse is credited only with the time it saves beyond them.

4 Experimental Methodology

This section describes the setup shared by all experiments: the machine and software (Section 4.1), the videos and prompts (Section 4.2), the dense reference (Section 4.3), how a configuration is run and scored (Section 4.4), and how time is measured (Section 4.5). Dense SAM 3.1, Eventful and EP-ViT are all run through the same harness, so they see the same sessions, frames, preprocessing and evaluator.

4.1 Hardware and Software

All experiments run on one workstation with four NVIDIA RTX A5000 GPUs (Table 4.1). Each configuration runs on a single GPU, and no two configurations share a GPU. The A5000 is power-capped at 230 W, and its clocks cannot be locked without root access, so its clock varies with temperature and load (Section 4.5). The machine is shared with another group, whose CPU jobs change the host load during our runs. This affects host-side timings but not GPU timings.

4.2 Dataset

Benchmark. We evaluate on the SA-V validation split of SA-Co/VEval [3], the video benchmark released with SAM 3. Each sample is a (video, noun phrase) *pair*. The model receives the phrase once and must find, segment and track every instance of it. In a *positive* pair the phrase is present, and the ground truth is one masklet per instance; in a *negative* pair it is absent and the model should predict nothing. The SA-V videos are recorded at 24 Frames Per Second (FPS) with resolutions from 480×848 to 1920×1080, and masks are annotated on every fourth frame. The frames were obtained from a public mirror of the benchmark, and every file was checked against the archive's checksums.

Table 4.2: Evaluation subsets of the SA-Co/VEval SA-V validation split. Model steps = native 24 fps frames fed to the pipeline over all pairs of the subset.

Subset	Videos	Ann. frames	Pairs	Pos. / neg.	GT masklets	Model steps
`normal`	10	983	73	48 / 25	129	29 281
`high_motion`	10	936	125	52 / 73	138	50 513
Total	20	1919	198	100 / 98	267	79 794

Subsets. The full split has 146 videos and 1504 pairs. Running every configuration on all of them at 24 FPS was not feasible, so we use two fixed subsets of ten videos each, chosen by a motion statistic before any reuse experiment was run. For each video we compute the mean fraction of cells of the 72×72 token grid whose mean absolute pixel difference between consecutive annotated frames exceeds 8 grey levels. Among videos with at least two positive phrases and 50–150 annotated frames, `normal` contains the ten with the least motion (all in the bottom quartile, 6–18 % of cells changed) and `high_motion` the ten with the most (top quartile, 77–89 %). Table 4.2 summarises them. The subsets are the two ends of the motion range: `normal` is best read as *least motion*. `high_motion` has many more negative pairs, which makes the presence term of cgF_1 more influential there.

Frame rate. The official protocol feeds the model only the annotated frames, i.e. 6 FPS. Temporal reuse depends on how much consecutive frames differ, and a camera or a streaming application delivers every frame, so we run at the native 24 FPS: the model sees every frame from frame 0 to the last annotated one, and only the annotated frames (native frame $4j$ = annotated frame j) are scored. This quadruples the number of model steps.

Preprocessing. Frames are loaded with SAM 3.1's own video loader: each JPEG is resized to 1008×1008 without keeping the aspect ratio and normalised with mean and standard deviation 0.5. The frames are held in host memory in fp16 and moved to the GPU one at a time. For EP-ViT, the same JPEG frames are additionally encoded to H.264 offline (Section 6.1).

4.3 Dense Baseline

Model. The reference is the released SAM 3.1 multiplex video predictor with its official checkpoint, built with SAM 3.1's own model builder. Inference runs under bf16 autocast with TF32 enabled, as the predictor sets it up, inside `torch.inference_mode`. FlashAttention-3 is disabled because it is not supported on Ampere. The detector and postprocessing run at batch size 1, with a hot-start delay of 15 frames. The neck, detector, tracker and memory are identical in every configuration; only the trunk differs.

Free optimisations. Before any reuse experiment we applied two optimisations to the dense trunk that require no approximation, so that neither method is credited with them: the linear weights of the 32 blocks are stored in bf16, which removes a cast on every call, and the rotary embedding, which the stock code runs as 22 small kernels per block, is compiled into one. Compiling the whole trunk gives a further gain. This yields three dense presets:

- **B0**: the stock SAM 3.1 trunk;
- **B1**: B0 with bf16 weights and compiled RoPE, otherwise eager. This is the eager dense baseline, and the model on which all reuse variants are built;

- **B2d**: B1 with the whole trunk compiled by `torch.compile` (`mode="default"`). This is the compiled dense baseline.

In an interleaved trunk benchmark (Section 4.5) the trunk takes 135.9 ms per frame for B0, 101.9 ms for B1 and 92.5 ms for B2d. B1 and B2d compute the same function as B0 up to kernel-level rounding.

Fair comparison. Compilation alone makes the dense trunk 8–9 % faster, so a compiled reuse method is compared only with compiled dense B2d, and an eager one only with eager B1. Every reported speedup states its reference. Dense runs are repeated in each experiment with that experiment's harness, rather than copied from another experiment.

4.4 Evaluation Protocol

Sessions. Each pair is one inference session: the video is loaded, the noun phrase is added as a text prompt on frame 0, the pipeline propagates forward through the video, and the session is closed. There are no interactive corrections and no backward propagation. All reuse state is reset at the start of every session, so every pair begins with a flush and no cache is carried from one phrase or video to the next.

Configurations and repeats. Each configuration is one pass over all 198 pairs. Inference is deterministic up to GPU kernel scheduling, so no random seeds are involved. Each run begins with one discarded warm-up session on the first video, long enough to exercise both the flush and the sparse path, and every compiled graph is built before the first timed frame.

Scoring. Predictions are written in the benchmark's format, one masklet per object identity with one mask per annotated frame, and scored with SAM 3's official SA-Co/VEval evaluator. The ground truth is restricted to the evaluated pairs. We report cgF_1 and pHOTA (Section 7.1) on each subset and on both subsets together.

Failures and common pairs. At 24 FPS a few sessions fail for reasons unrelated to reuse: an out-of-memory error on the most crowded long pair, and a crash inside the upstream tracker. Failed pairs are recorded and excluded. Methods are compared only on the pairs that every compared run completed; for the main comparison these are 189 of the 198 pairs.

Agreement with dense. Besides the benchmark metrics, we measure how far a method moves the pipeline's output: the mean IoU, per annotated frame, between the union of its predicted masks and that of the dense run. Running the dense pipeline with a different but exact kernel configuration (B0 versus B1) gives the agreement expected from rounding alone, against which the reuse methods are judged.

4.5 Latency and Profiling Methodology

Pipeline timing. During the evaluation runs, time is recorded with CUDA events, which the GPU timestamps when it reaches them in the stream. Events are recorded around every trunk call (by hooks on the module installed as the neck's trunk) and around every per-frame model step, and they are read once at the end of each session, so timing adds no synchronisation to the run. The per-frame step contains the trunk, neck, detector, tracker and memory encoder, but not JPEG decoding or output postprocessing. Section 7.3 defines the reported quantities.

Interleaved trunk benchmark. Pipeline runs of different configurations take hours and run at different times on a shared, power-capped GPU, so their latencies are affected by the GPU's temperature and clock and by host load. For clean trunk comparisons we use a separate benchmark that runs every configuration in one process. Each frame is passed through every configuration in rotating order, after a 90 s warm-up that brings the GPU to its steady temperature, so all configurations see the same clock. Only steady (non-flush) frames are timed. GPU clock, temperature and power are sampled throughout. Without interleaving, the trunk time of the same configuration drifts by about 4 %; with it, the dense presets reproduce across sessions to within about 1 %.

Controlled end-to-end measurement. The end-to-end step contains host-side work, so it cannot be compared across runs made at different times (Section 7.4). To measure the end-to-end gain of EP-ViT we therefore ran one pair from each of the 20 videos (7616 steps), in four rounds of three processes on three idle GPUs. The three processes of a round started together and ran the same pairs, so they saw the same host load. Each round contained the dense pipeline and two EP-ViT settings: dense, D and F; A, dense and C; B, E and dense; dense, G and H. A speedup is taken against the dense run of the same round, and the four dense runs give the noise floor.

Profilers. Kernel-level behaviour is measured with Nsight Systems. Our code marks frames, blocks and gate operations with NVTX ranges, and every kernel is attributed to the range that launched it, which gives the GPU time per block and per operation and the idle gaps between kernels. The per-frame spans agree with the CUDA-event times to within 0.1–0.4 ms, and the per-range times agree with `torch.profiler` to within 0.5–2.6 %. For EP-ViT we captured 20 consecutive frames, starting at frame 40, of one pair per class for dense, 75 % and 50 %. Memory is measured with `torch.profiler`'s allocator events, which attribute every allocation to a range. Nsight Compute, which needs root access on this machine, was used once to measure DRAM traffic and bandwidth of the gate kernels, with `-clock-control none` so that it did not change the GPU's clocks for other users.

MAC counting. Executed MACs are computed from each method's per-frame log of processed rows (Section 7.2). The counters were validated against PyTorch's Floating-Point Operation (FLOP) counter for every configuration. The FLOP counter changes the numerics of the compiled RoPE for the rest of the process, so it is never run in a process that is timed or whose outputs are compared.

5 Eventful Transformer Implementation

5.1 Integration into SAM 3.1

[TODO: Integration into SAM 3.1.]

5.2 Token Gating and Feature Caching

[TODO: Token Gating and Feature Caching.]

Algorithm 1 TODO: Eventful token gating (skeleton).

Require: TODO
Ensure: TODO
1: **[TODO: fill in algorithm]**

5.3 Layer-Selective Execution

[TODO: Layer-Selective Execution.]

5.4 Implementation Optimizations

[TODO: Implementation Optimizations.]

6 Codec-Guided EP-ViT

The second method replaces the SAM 3.1 ViT trunk with an implementation of Encoding-Prior Vision Transformer (EP-ViT) [2]. Unlike the Eventful gates, which measure change in the network's own activations, EP-ViT decides which tokens to recompute before the encoder runs, from the motion vectors and residuals of an H.264 bitstream [8]. The paper releases no code, so the implementation below is our own. It is training-free: the trunk keeps SAM 3.1's weights unchanged, and the neck, detector, tracker and memory are not modified. Section 6.1 describes how the codec information is produced, Section 6.2 how it is mapped onto the token grid, Section 6.3 how the encoder recomputes some tokens and reuses the others, and Section 6.4 how the method is integrated into the video pipeline. Figure 6.1 shows the whole system.

6.1 Codec Motion Vectors and Residuals

Encoding. The evaluation videos are stored as JPEG frames, so we re-encode them with `libx264` the way a camera would, in the source resolution (padded to even), 4:2:0 chroma and at the native 24 FPS. The encoder settings are chosen so that the bitstream can be read without ambiguity:

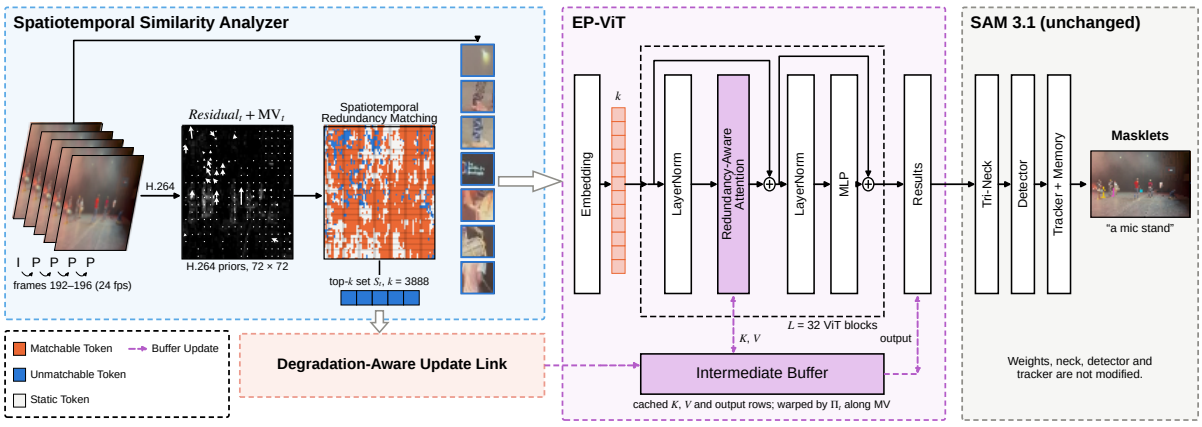


Figure 6.1: EP-ViT in the SAM 3.1 pipeline, with real data from `sav_012075` (frames 192–196). The Spatiotemporal Similarity Analyzer turns the H.264 residuals and motion vectors into token classes on the 72×72 grid and selects the top-k set S_t. The trunk computes those tokens and reuses the others from the Intermediate Buffer, whose rows are warped along the motion vectors. The Degradation-Aware Update Link forces tokens with accumulated error back into S_t. The neck, detector and tracker are unchanged.

- `-bf 0`: only I- and P-frames, as in the paper, so every frame predicts from the past;
- `-refs 1`: every P-frame predicts from the previous frame only. The exported motion vectors do not say which reference frame a block used, so with a single reference a vector always points into frame $t-1$;
- `-g` G `-keyint_min` G `-sc_threshold 0`: an I-frame exactly every G frames and no scene-cut I-frames, so the refresh pattern is fixed and known in advance;
- `-crf` C: constant quality.

Unless stated otherwise we use $C = 23$ and $G = 192$, i.e. one I-frame every 8 s of video.

Extraction. Each encoded video is decoded once with PyAV, with the decoder's motion-vector export enabled. For every P-frame the decoder returns a list of blocks, each with its size, position and displacement into the previous frame. These are rasterised into a per-pixel displacement field $\mathbf{d}_t(x, y)$; pixels that no block covers are intra-coded. The frame type is read with `ffprobe`, because the PyAV version we use reports every I-frame after the first as a P-frame.

The decoder does not expose the coded residual. We therefore reconstruct it from decoded luma as what remains after motion compensation:

$$r_t(x, y) = \left|Y_t(x, y) - Y_{t-1}(x + d_t^x(x, y),\ y + d_t^y(x, y))\right|, \tag{6.1}$$

with nearest-pixel sampling and clamping at the frame border. This is the quantity the codec's residual encodes (Eq. 3 of the paper), measured in luma levels (0–255).

All of this runs offline on the CPU, once per video and encoder setting, and is stored as a compressed array per video. Encoding, motion-vector export and token mapping together take about 160 ms per frame on our host. A deployment that receives an H.264 stream gets the motion vectors from the decoder it already runs, so we treat the priors as available and exclude this cost from all encoder and end-to-end timings.

6.2 Mapping Codec Information to ViT Tokens

Macroblocks to patches. SAM 3.1 stretches every $W \times H$ frame to 1008×1008 pixels and embeds it with 14×14 patches, giving a 72×72 grid of $N = 5184$ tokens. Token (i, j) therefore covers source rows $[iH/72,\ (i+1)H/72)$ and columns $[jW/72,\ (j+1)W/72)$, a region that does not align with the codec's 16×16 macroblocks or their sub-partitions. Following the paper's MB-token matching (Eq. 12), each token takes the *majority* motion vector of the pixels it covers. We sample the displacement field on a 4×4 grid, the size of the smallest H.264 partition, quantise each sample to quarter-pel, and keep the most frequent vector within the token. The vector is then converted to token units, $\mathbf{v}_t(i, j) = (d^x \cdot 72/W,\ d^y \cdot 72/H)$, because the stretch to 1008^2 scales the two axes differently. Per token we also store the mean residual $\bar{r}_t(i, j)$ of Eq. (6.1) and the share $q_t(i, j)$ of its pixels that are intra-coded.

Sign convention. $\mathbf{v}_t(i, j) = (v^x, v^y)$ points from the token's position in frame t to where its content was in frame $t-1$. The *reuse source* of token (i, j) is therefore the token $\mathrm{src}(i, j) = (i + \mathrm{round}(v^y),\ j + \mathrm{round}(v^x))$.

Token classes. On a P-frame each token is put into one of the paper's three classes (Eq. 6):

$$\begin{aligned} \text{static:} \quad & \mathbf{v} = \mathbf{0}, \quad \bar{r} < \tau_1, \quad q \le q_{\max}, \\ \text{matchable:} \quad & \mathbf{v} \neq \mathbf{0}, \quad \bar{r} < \tau_2, \quad q \le q_{\max}, \quad \mathrm{src}(i,j) \text{ inside the grid}, \\ \text{unmatchable:} \quad & \text{otherwise.} \end{aligned} \tag{6.2}$$

A static token's content has not moved, so its cached features can be kept in place. A matchable token's content has moved but is still predicted well by the previous frame, so its features can be taken from its source token. An unmatchable token is new content (a large residual, mostly intra-coded, or moved in from outside the frame) and must be recomputed. We use $\tau_1 = 4$, $\tau_2 = 6$ luma levels and $q_{\max} = 0.5$. On an I-frame every token is unmatchable.

6.3 Token Selection and Feature Reuse

Figure 6.2 summarises how the priors become a token plan on a P-frame: the classes of Section 6.2, the update link, the choice of k tokens and the reuse of the rest. The paragraphs below describe each of these steps.

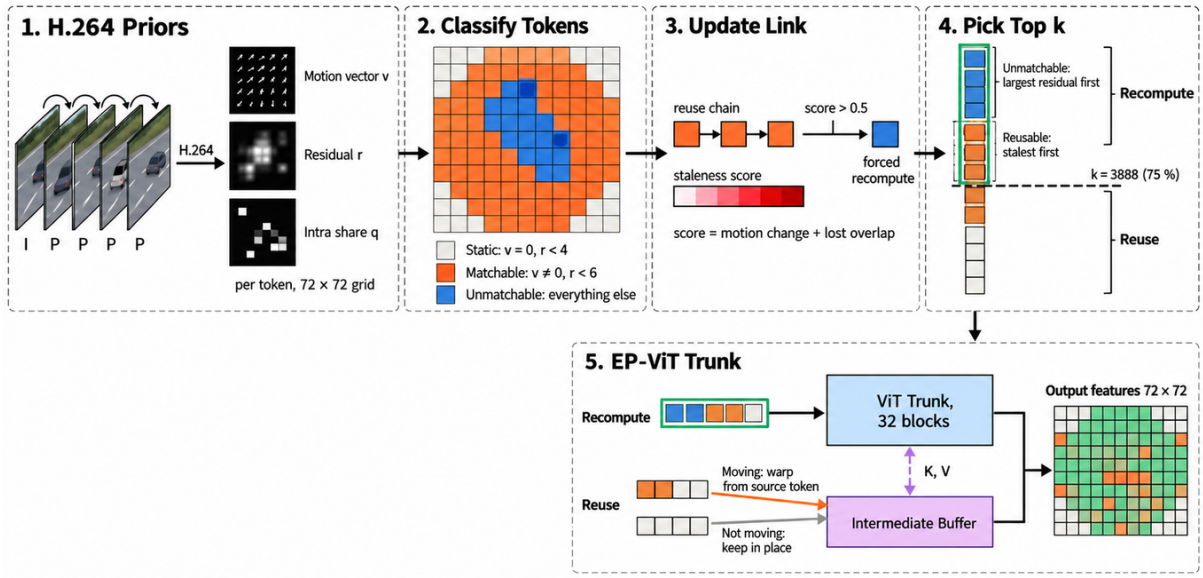


Figure 6.2: From H.264 priors to the token plan of one P-frame. The figure is schematic: the frames are illustrative and the 12×12 grid stands for the real 72×72 grid. I-frames and the first frame of a session skip these steps and recompute all 5184 tokens.

Degradation-aware update link. A token that is reused frame after frame accumulates error: each warp moves features by a whole token while the true motion is fractional, and the motion can change along the chain. The paper's update link (Eq. 11) tracks this per token. Let $\mathbf{f} = \mathbf{v} - \mathrm{round}(\mathbf{v})$ be the fractional part of a token's motion. The overlap between the token cell shifted by $\mathbf{f}$ and the grid cell it is assigned to is

$$\mathrm{IoU}(\mathbf{f}) = \frac{a}{2 - a}, \qquad a = (1 - |f^x|)(1 - |f^y|), \tag{6.3}$$

and it is accumulated multiplicatively along the reuse chain, $\mathrm{iou}_t(j) = \mathrm{iou}_{t-1}(\mathrm{src}(j)) \cdot \mathrm{IoU}(\mathbf{f}_t(j))$. The degradation score is

$$s_t(j) = \lambda \left\| \mathbf{v}_t(j) - \mathbf{v}_{t-1}(\mathrm{src}(j)) \right\|^2 + \left(1 - \mathrm{iou}_t(j)\right), \tag{6.4}$$

and a static or matchable token with $s_t(j) > \theta$ is forced back to recomputation. We use $\lambda = 1$ and $\theta = 0.5$. Recomputing a token resets its chain (iou = 1).

Token budget. The paper recomputes exactly the unmatchable and forced tokens, so the cost varies from frame to frame with the video content. We instead fix a budget of k tokens per P-frame. Tokens are ranked by priority: unmatchable and forced tokens first, in order of decreasing residual, then the remaining reusable tokens in order of decreasing degradation score s_t. The k highest-ranked tokens are recomputed. If fewer than k tokens need recomputation, the rest of the budget goes to the reused tokens that have degraded most; if more need it, the ones with the smallest residual are reused anyway. A fixed k gives every P-frame the same tensor shapes, which the compiled implementation needs (Section 6.4), and makes the compute per frame a parameter rather than an outcome. The paper's variable rule is also implemented (rounded up to a multiple of 128 tokens) but not used in the experiments.

Feature reuse. Tokens that are not recomputed take their features from the cache. A token that has not moved keeps its own cached row. A moving token with a source inside the grid is *warped*: before the blocks run, its cached rows are copied from its reuse source. This includes unmatchable tokens that miss the budget. The cache holds, for each of the 32 blocks, the keys and values of all N tokens, and the trunk's output features. For a warped token all of these are copied (a gather followed by a scatter).

Sparse forward pass. Patch embedding, the absolute position embedding and the first layer norm run on the whole frame (0.1 % of the trunk's MACs). The $M = k$ selected rows are then gathered, and every block runs its layer norms, QKV projection, output projection and Multi-Layer Perceptron (MLP) on these M rows only. The new keys and values of the M rows are written into the block's N-row buffers, which now hold current values for the recomputed tokens and cached or warped values for the rest. In attention, only the M query rows are needed downstream, so the paper's sparse update of $QK^\top$ and AV (Eqs. 8–10) reduces to the selected queries attending to all buffered keys and values:

$$\mathbf{o}_i = \mathrm{softmax}\left(\frac{\mathbf{q}_i K^\top}{\sqrt{d}}\right) V, \qquad i \in \mathcal{S}, \tag{6.5}$$

where $\mathcal{S}$ is the set of selected tokens and K, V are the full buffers. Given the buffers, this is exact for the selected rows. In the four global blocks (7, 15, 23 and 31) this is an $M \times N$ attention. In the 28 windowed blocks each 24×24 window attends with its own selected queries to its 576 keys; the queries of all nine windows are padded to the largest per-window count m, so that the nine windows form one batched attention call. After block 31 the M output rows are scattered into the N-row output buffer, and the trunk returns this buffer as the $(1, 1024, 72, 72)$ feature map.

Positional encoding of reused tokens. SAM 3.1 applies rotary position embeddings (RoPE) [18] inside attention. The key buffers therefore store keys *before* RoPE, and RoPE is

Algorithm 2 One EP-ViT trunk call (frame t, budget k).

Require: frame x_t, priors $(\mathbf{v}_t, \bar{r}_t, q_t)$, buffers K_l, V_l $(l = 1..32)$, output buffer O

if first frame of the session, I-frame, or $t \neq t_{\text{last}} + 1$ **then**
  $\mathcal{S} \leftarrow$ all N tokens; reset all reuse chains ▷ flush: dense forward
else
  classify tokens (Eq. (6.2)); compute scores s_t (Eq. (6.4))
  $\mathcal{S} \leftarrow$ the k highest-priority tokens
  $\mathcal{W} \leftarrow$ moving tokens not in $\mathcal{S}$ with $\text{src}(j)$ inside the grid
  copy rows $\text{src}(j) \to j$ of every K_l, V_l and of O, for $j \in \mathcal{W}$
end if
$h \leftarrow \text{gather}_{\mathcal{S}}(\text{LN}(\text{PatchEmbed}(x_t) + \text{pos}))$
for $l = 1, \ldots, 32$ **do**
  $q, k, v \leftarrow \text{QKV}_l(\text{LN}(h))$; $K_l[\mathcal{S}] \leftarrow k$; $V_l[\mathcal{S}] \leftarrow v$
  $h \leftarrow h + \text{Proj}_l(\text{Attn}(\text{RoPE}(q), \text{RoPE}(K_l), V_l))$ ▷ global or per window
  $h \leftarrow h + \text{MLP}_l(\text{LN}(h))$
end for
$O[\mathcal{S}] \leftarrow h$; **return** O as a $(1, 1024, 72, 72)$ feature map

applied to the whole key buffer at every frame, so a warped key is rotated for its new position. The selected queries are rotated with the frequencies of their own positions. The tiled absolute position embedding, in contrast, is added once before the first block, so a warped token's cached features still carry the absolute embedding of its source position. Correcting this would require recomputing the token, which defeats the purpose of reuse, so we leave it uncorrected.

6.4 Gating and Implementation

Gated operations. In the method of Algorithm 2 the residual stream itself has only M rows, so every operation of every block (QKV projection, attention queries, output projection and MLP) is gated by the same token set. To compare with Eventful, which gates only the MLP, we also implement a selective variant. It keeps the full N-row residual stream and an N-row output buffer for each gated operation. On a P-frame a gated operation recomputes only the k selected rows and writes them into its buffer, whose other rows keep their cached (or warped) values. Attention, the layer norms and every operation that is not gated run densely on all N rows. This variant can gate any subset of {QKV, output projection, MLP} in any subset of blocks. The selection of tokens, the warping and the refresh policy are identical in both variants.

Refresh policy. A frame is encoded densely (a *flush*) when it is the first frame of a session, when it is an I-frame, and when it is not the successor of the previously encoded frame, since the motion vectors of frame t only describe the step from $t - 1$. A flush runs the same code path with all N tokens selected, rebuilds every buffer, and resets the reuse chains, so with $G = 192$ no token is reused for more than 8 s. With all tokens selected the computation is that of the original trunk; a unit test checks this against SAM 3.1's ViT.

Integration into SAM 3.1. SAM 3.1 owns a single ViT, held as the `trunk` attribute of the neck that feeds both the detector and the tracker. EP-ViT wraps this ViT, uses its modules and weights directly, and is installed by replacing that attribute, so it serves every image encode of the pipeline. Three hooks on the model instance connect it to the video pipeline without changing SAM 3.1's classes:

- resetting the inference state also resets EP-ViT, so every prompt session starts with a flush;
- initialising the inference state records a fingerprint of the video's preprocessed frames (a strided sample of their pixels);
- a forward pre-hook on the trunk matches each incoming frame against this fingerprint to obtain its frame index, which selects the codec priors. This costs one small device-to-host copy per trunk call.

The pipeline encodes frames 0 and 1 once for the prompt and again when propagation starts. A frame that was already encoded in the current session is served from a small output cache (the last four frames) and leaves the reuse state unchanged.

Host scheduling. The classification, update link and selection run on the CPU in NumPy, one step per encoded frame, so the scheduler state always matches the buffers on the GPU. The resulting plan (the selected token indices, the warp sources and destinations, and the window layout of the padded queries) is moved to the GPU in a single pinned host-to-device copy, and all tensor sizes are known on the host, so the sparse forward pass needs no synchronisation.

Compilation and memory. For timing, the per-frame GPU work (patch embedding, gather, the 32 blocks and the final scatter) is compiled with `torch.compile`, with the same settings as the compiled dense and Eventful runs (`mode="default"`, `dynamic=False`). Shapes are static per pair (M, m): M is the budget k (or N on a flush), and the padded per-window query count m is rounded up to a multiple of 64. This gives at most nine sparse graphs and one flush graph, all compiled before anything is timed. The padding rows are zero queries whose outputs are discarded. Only the host plan, the variable-length warp copies and the output write stay outside the graph. The key and value caches are allocated as one tensor per block rather than a single $32 \times N$ tensor: with a single tensor, the compiler implemented each write into one block's slice as a copy of the whole tensor, and the compiled trunk was slower than the uncompiled one. In bfloat16 the key and value caches of the 32 blocks take about 680 MB together ($2 \times 32 \times 5184 \times 1024$ values of 2 bytes), and the output buffer 10.6 MB.

Executed compute. MACs are counted from the per-frame log of selected rows. With $C = 1024$ channels, MLP width $C_f = 4736$, M selected tokens and m padded queries per window, one frame of the full method executes

$$32\,M\,(4C^2 + 2CC_f) \;+\; 4 \cdot 2MNC \;+\; 28 \cdot 2 \cdot 9\,m \cdot 576\,C \;+\; N \cdot 3 \cdot 14^2 \cdot C \tag{6.6}$$

MACs: the linear layers of 32 blocks, attention in the 4 global and 28 windowed blocks, and the patch embedding. For $M = N$ and $m = 576$ this is the dense trunk's 2699 GMAC. A frame served from the output cache executes none. The counts were checked against PyTorch's FLOP counter.

7 Evaluation Metrics

Every configuration is described by three kinds of quantity: the accuracy of the full pipeline (Section 7.1), the computation the encoder actually executes (Section 7.2), and the time it takes

(Section 7.3). Section 7.4 defines how methods are compared with the dense baseline and how we decide whether a difference is larger than chance.

7.1 cgF1 and pHOTA

Both metrics are computed by SAM 3's official SA-Co/VEval evaluator [3] on mask predictions. A prediction is a masklet: one object identity with a mask on every annotated frame of the pair and one confidence score, which we set to the mean presence probability of the object over the scored frames. Only predictions with a score above 0.5 are evaluated, so both metrics judge the pipeline's hard decisions rather than a ranking. Every (video, noun phrase) pair is evaluated as a separate sequence, so predictions for one phrase are never matched to the ground truth of another. All scores are reported $\times 100$.

Masklet IoU. The overlap between a predicted masklet d and a ground-truth masklet g is a spatio-temporal IoU, accumulated over the annotated frames τ of the pair before dividing,

$$\mathrm{IoU}(d, g) = \frac{\sum_\tau |d_\tau \cap g_\tau|}{\sum_\tau |d_\tau \cup g_\tau|}. \tag{7.1}$$

A prediction that drifts to another object or disappears halfway through the video therefore loses overlap on every frame where it is wrong.

cgF$_1$. The concept-grounded F_1 score separates two questions: is the concept present in the video at all, and if so, are its instances found and segmented? Within each pair, predictions and ground-truth masklets are matched one-to-one by the Hungarian algorithm on Eq. (7.1). At an IoU threshold α, a matched prediction with $\mathrm{IoU} \geq \alpha$ is a true positive, every other prediction a false positive, and every unmatched ground-truth masklet a false negative. Counts are summed over pairs (micro-averaging), with false positives taken only from positive pairs on which the model predicted something, which gives the *positive micro* F_1

$$\mathrm{pmF}_1(\alpha) = \frac{2\,P(\alpha)\,R(\alpha)}{P(\alpha) + R(\alpha)}, \qquad P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}_+}, \quad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}. \tag{7.2}$$

Presence is judged per pair: a pair is a true positive if the phrase is present and the model predicts at least one masklet, a false negative if it predicts none, a false positive if the phrase is absent and the model predicts something, and a true negative otherwise. The image-level Matthews correlation coefficient of these counts is

$$\mathrm{IL_MCC} = \frac{\mathrm{TP}_p\,\mathrm{TN}_p - \mathrm{FP}_p\,\mathrm{FN}_p}{\sqrt{(\mathrm{TP}_p + \mathrm{FP}_p)(\mathrm{TP}_p + \mathrm{FN}_p)(\mathrm{TN}_p + \mathrm{FP}_p)(\mathrm{TN}_p + \mathrm{FN}_p)}}, \tag{7.3}$$

and cgF_1 is the product of the two, averaged over the ten thresholds $\alpha \in \{0.50, 0.55, \ldots, 0.95\}$:

$$\mathrm{cgF}_1 = \frac{1}{10} \sum_\alpha \mathrm{pmF}_1(\alpha) \cdot \mathrm{IL_MCC}. \tag{7.4}$$

A model that segments well but hallucinates absent concepts is penalised through IL_MCC, and one that detects presence well but segments poorly through pmF_1. IL_MCC depends on only about one decision per pair (198 on our subsets), and a single flipped decision moves it by about a point, so cgF_1 is a coarse measure at our subset sizes.

pHOTA. Phrase HOTA applies Higher Order Tracking Accuracy [22] to every (video, phrase) pair as a separate sequence, with all phrases treated as one class. HOTA matches predictions to ground truth on every annotated frame at a localisation threshold α and scores detection and association jointly:

$$\mathrm{HOTA}_\alpha = \sqrt{\mathrm{DetA}_\alpha \cdot \mathrm{AssA}_\alpha}, \qquad \mathrm{DetA}_\alpha = \frac{|\mathrm{TP}|}{|\mathrm{TP}| + |\mathrm{FN}| + |\mathrm{FP}|},$$
$$\mathrm{AssA}_\alpha = \frac{1}{|\mathrm{TP}|} \sum_{c \in \mathrm{TP}} \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|}. \tag{7.5}$$

Here TP, FN and FP are per-frame mask detections, and for a true positive c, $\mathrm{TPA}(c)$ counts the frames on which the same prediction and ground-truth pair is matched, while $\mathrm{FNA}(c)$ and $\mathrm{FPA}(c)$ count the frames on which either of the two is matched to something else or to nothing. Counts are combined over all pairs, and pHOTA is the mean of HOTA_α over $\alpha \in \{0.05, 0.10, \ldots, 0.95\}$. Because it is computed from every annotated frame of every object, pHOTA is far less coarse than cgF_1 and is our primary accuracy metric. It is also sensitive to the errors temporal reuse is expected to cause: a mask that degrades on some frames lowers DetA, and an identity that switches or is lost lowers AssA. Negative pairs affect pHOTA only through the false positives predicted on them.

7.2 Executed MACs

Computation is measured as the MACs the encoder actually executes, not the MACs it would execute at the nominal budget. Each method logs, for every trunk call, how many rows each operation processed, and the count is derived from that log:

- a linear layer (QKV projection, output projection, the two MLP layers) that processes M rows costs $M \times d_{\text{in}} \times d_{\text{out}}$ MACs;
- attention costs $2 \times (\text{queries}) \times (\text{keys}) \times D$ for the queries actually processed, including the padding rows of EP-ViT's windowed attention;
- the patch embedding always runs on all N tokens;
- a flush frame is counted as dense, and a frame served from EP-ViT's output cache as zero.

The reported value is the mean over all frames passed to the trunk, including the frames that the pipeline encodes twice, so it reflects flushes and refreshes as they actually occurred. The neck, detector and tracker are not counted, because no method changes them. The dense trunk executes 2699 GMAC per frame (Table 2.1), and we report the reduction $1 - C/C_{\text{dense}}$ (Eq. (3.3)).

The work a method spends on deciding what to recompute is not counted: the change scores and top-k selection of Eventful, EP-ViT's host-side scheduler and codec processing, and the gather, scatter and cache copies of both. This work has almost no multiply–accumulates, but it costs time, so it appears in the latency (Section 7.3) and is analysed in Section 9. The MAC counter of every configuration was checked against PyTorch's FLOP counter.

7.3 Encoder and End-to-End Latency

Encoder latency. The encoder latency is the time of the ViT trunk per frame on the GPU stream. CUDA events are recorded immediately before and after every call of the module installed as the neck's trunk (Section 4.5), so it includes everything the reuse method does inside

that call: gate scores and selection for Eventful; the scheduler step, plan upload and cache warps for EP-ViT; and the gathers, scatters and cache writes of both. Per run, the encoder latency is the total trunk time divided by the number of frames encoded, including flushes and the repeated encodes of the first frames. It excludes the neck and everything after it.

End-to-end latency. The end-to-end (E2E) latency is the time of one per-frame model step of the pipeline, measured with CUDA events around the step: the trunk, the neck, the detector, the tracker's memory attention and mask decoder, and the memory encoder. It excludes JPEG decoding, which happens before the step, and the postprocessing of outputs into the benchmark format, which is not part of inference. We report the median over all propagation steps of a run, which is robust to occasional outlier steps. EP-ViT's frame-index lookup falls inside the E2E step but outside the encoder time.

What is not timed. The codec work of EP-ViT (H.264 encoding, motion-vector export and token mapping) is done offline and is not part of either latency. We assume a deployment that receives an H.264 stream and obtains motion vectors from the decoder it already runs, and report the offline cost separately. Model loading, compilation and the warm-up session are also excluded.

7.4 Speedup and Statistical Analysis

Speedup. For a method m and a dense reference d run with the same compile setting (Section 4.3), the encoder and end-to-end speedups are

$$S_{\text{enc}} = \frac{T^d_{\text{enc}}}{T^m_{\text{enc}}}, \qquad S_{\text{e2e}} = \frac{T^d_{\text{e2e}}}{T^m_{\text{e2e}}}. \tag{7.6}$$

To relate time to computation (RQ2), we also report what fraction of the removed MACs is removed as time,

$$\eta = \frac{1 - T^m_{\text{enc}}/T^d_{\text{enc}}}{1 - C^m/C^d}, \tag{7.7}$$

which is 1 if encoder time falls in proportion to executed MACs, below 1 if the overheads of Section 3.3 eat part of the saving, and negative if the method is slower than dense.

Timing variability. Encoder times are GPU timestamps and are robust to host load: the same configuration reproduces to within 1–1.5 % between runs, and to within about 1 % in the interleaved benchmark. E2E times are not. The step contains host-side work, such as launching the detector and tracker and their small host–device copies, and our host is shared with another group's jobs. Two runs of the identical dense pipeline differed by 4 % in median E2E latency. We therefore report the E2E speedups of EP-ViT from a dedicated measurement in which dense and EP-ViT run at the same time (Section 4.5), where four repeats of the dense pipeline agree to within 1.2 %.

Accuracy differences. Each configuration is run once on a fixed set of pairs, so the main source of uncertainty in a metric difference is which pairs happen to be in the subset. We estimate it with a paired bootstrap. The pairs are resampled with replacement $B = 200$ times; on each resample, the dense run and the method are both scored with the official evaluator, and the difference is recorded. The middle 95 % of the B differences is the 95 % confidence

Table 8.1: Dense SAM 3.1 at 24 fps on the common pairs. Accuracy ×100. Encoder = trunk time per frame; E2E = median per-frame model step; rest = E2E minus encoder; memory = median over pairs of the per-pair peak GPU allocation. Both presets execute 2699 GMAC per frame in the trunk.

Preset	Scope	Pairs	cgF_1	pHOTA	Encoder (ms)	E2E (ms)	Rest (ms)	Memory (GiB)
B1 (eager)	All 20 videos	189	12.04	48.92	98.2	185.5	87.3	–
	`normal`	70	13.69	55.35	98.6	177.4	78.8	5.14
	`high_motion`	119	11.16	38.56	97.9	190.6	92.7	6.79
B2d (compiled)	All 20 videos	189	12.41	48.88	89.9	175.0	85.1	–
	`normal`	70	15.43	55.45	89.4	172.7	83.3	5.13
	`high_motion`	119	11.00	38.35	90.2	177.6	87.4	6.77

interval, and a difference is called significant only if this interval excludes zero. Pairing removes the variation that both runs share, which is most of it. As a check, the interval for two exact dense configurations (B0 against B1) contains zero on every subset. With our subset sizes the intervals can only exclude losses larger than about 2 points of pHOTA and 3–5 points of cgF_1; smaller differences are not interpreted.

Agreement with dense. Benchmark metrics can stay unchanged while the output changes, because errors may cancel or fall on objects that were already wrong. We therefore also report how far each method moves the pipeline's output, as the mean, over annotated frames, of the IoU between the union of its predicted masks and that of the dense run. Its reference value is the agreement between two exact dense configurations (B0 against B1), which measures kernel-level rounding alone.

8 Experimental Results

8.1 Dense Baseline

Table 8.1 gives the dense reference on the 189 pairs that every run in this section completed. The nine excluded pairs failed in at least one run, through the upstream tracker crash or, on one crowded long pair, an out-of-memory error, in the dense runs as well as the others.

Reproducibility. The eager baseline B1 was run twice, once for the Eventful experiments and once with the harness used for EP-ViT. All 198 pairs produced byte-identical predictions in the two runs, so the harness itself adds no variation, and every later difference between configurations comes from the encoder.

Accuracy. Over all 20 videos dense SAM 3.1 reaches 48.9 pHOTA and 12.0–12.4 cgF_1. The subsets differ strongly: pHOTA is 55.4 on `normal` and 38.4–38.6 on `high_motion`, so fast motion is hard for the dense model already. B1 and B2d compute the same function and differ only in kernel-level rounding, yet their cgF_1 differs by 1.7 points on `normal` (13.69 against 15.43), while pHOTA moves by at most 0.21 points. This is the scale of cgF_1 differences that arise without any change to the algorithm, and it confirms that pHOTA is the more reliable of the two metrics on our subsets (Section 7.1).

Latency. The eager trunk takes 98.2 ms per frame and the compiled trunk 89.9 ms, a 1.09× speedup from compilation alone. The median pipeline step falls from 185.5 to 175.0 ms (1.06×). The trunk takes 51 % of the compiled step; the remaining 85 ms (neck, detector, tracker and memory) are the same in every configuration, so by Eq. (3.4) no encoder-only method can make the compiled step faster than about 2.1×. All reuse results below are compared with compiled B2d.

Memory. The median per-pair peak GPU allocation is 5.1 GiB on `normal` and 6.8 GiB on `high_motion`. The peak grows with the number of tracked objects and frames: the most crowded `high_motion` pair reaches 23.3 GiB of the GPU's 24 GiB, which is why one pair fails in every configuration.

8.2 Eventful Performance

[TODO: Eventful Performance.]

8.3 Eventful Ablations

[TODO: Eventful Ablations.]

8.4 EP-ViT Performance

We ran eight EP-ViT settings on all 198 pairs. A–F gate every operation of each block with budgets from 100 % down to 50 % of the 5184 tokens. G and H keep the 75 % budget but gate only the QKV projection (G) or only the MLP (H), using the selective variant of Section 6.4. All runs are compiled, and the priors are H.264 encodes at CRF 23 with an I-frame every 192 frames. Accuracy is scored on the 189 common pairs. Latency comes from the controlled measurement of Section 4.5.

End-to-end latency. Table 8.2 gives the gain on the two video classes. At a 75 % budget, EP-ViT shortens the pipeline step by 23.9 ms on `normal` (1.15×, 5.33 to 6.11 FPS) and by 22.3 ms on `high_motion` (1.13×, 5.32 to 6.03 FPS). At 50 %, the savings are 39.8 ms (1.27×) and 38.1 ms (1.25×). Across the ten videos of a class, the gain per pair stays between 1.14 and 1.18× (`normal`, 75 %), 1.11 and 1.15× (`high_motion`, 75 %), 1.26 and 1.35× (`normal`, 50 %) and 1.22 and 1.28× (`high_motion`, 50 %). The fixed budget gives almost the same gain on the two classes, even though they have very different motion, because the trunk does the same amount of work on every sparse frame.

The dense pipeline was repeated in each of the four rounds, and its median step varied by at most 1.2 % (185.9–187.6 ms on `normal`, 186.8–188.9 ms on `high_motion`), which sets the noise floor of the speedups. The rest of the step stays between 93.8 and 99.3 ms in every run, so nearly all of the gain is the trunk. With the trunk at 48 % of the dense step, Eq. (3.4) predicts 1.13× at 75 % and 1.24–1.25× at 50 %, which matches the measurement to within 0.04×. The absolute steps are longer than those of the sweep in Table 8.1 (about 187 against 175 ms), because the set of pairs differs and three pipelines shared the host; the speedups are the result to read from Table 8.2.

These gains replace the end-to-end speedups of the original sweep (1.27× and 1.48×), which we do not report. There, the dense run and the EP-ViT runs were made one to two days apart

Table 8.2: End-to-end latency of EP-ViT on each video class against compiled dense SAM 3.1. The three methods of a class ran at the same time on separate GPUs over the same ten videos at 24 fps, so they saw the same host load. Encoder = trunk time per frame; E2E = median per-frame model step; FPS = 1000 / E2E. GMAC and pHOTA are from the full evaluation (70 `normal` and 119 `high_motion` pairs).

Video class	Method	GMAC	pHOTA	Encoder (ms)	E2E (ms)	S_{enc}	S_{e2e}	FPS
`normal`	Dense B2d	2699	55.45	89.9	187.5	1.00×	1.00×	5.33
	EP-ViT 75 %	**2050**	**54.25**	**68.2**	**163.6**	**1.32×**	**1.15×**	**6.11**
	EP-ViT 50 %	**1396**	**53.83**	**54.0**	**147.8**	**1.66×**	**1.27×**	**6.77**
`high_motion`	Dense B2d	2699	38.35	89.9	188.1	1.00×	1.00×	5.32
	EP-ViT 75 %	**2053**	**38.88**	**68.1**	**165.8**	**1.32×**	**1.13×**	**6.03**
	EP-ViT 50 %	**1402**	**36.28**	**51.8**	**150.1**	**1.73×**	**1.25×**	**6.66**

under different host load, and the unchanged part of the step differed by up to 22 ms between them.

Where the time goes. Figures 8.1 and 8.2 show the Nsight Systems timeline of the three methods at the same scale. The trunk range shortens while the ranges after it keep their length, so each step ends earlier and the next one starts sooner. Figure 8.3 and Table 8.3 give the same comparison for a median frame. On `normal`, the trunk falls from 88.2 to 65.9 ms at 75 % and to 47.9 ms at 50 %, while the neck, detector, tracker and memory each change by less than 1 ms on both classes. The whole GPU span falls by 1.12 and 1.26×. This agrees with the end-to-end measurement, and it shows that the gain is real GPU time rather than a change in host load. Each capture covers 20 frames of one pair under the profiler, so its absolute times are not the reported latency.

Table 8.3: GPU time per frame by pipeline stage from Nsight Systems, in ms (median of 20 consecutive frames of one pair per class). Span = first kernel to last kernel of the step, including idle gaps between kernels, so it exceeds the sum of the stages.

Video class	Method	Trunk	Neck	Detector	Tracker	Memory	Span	Speedup
`normal`	Dense B2d	88.2	21.2	31.3	26.6	10.1	193.8	1.00×
	EP-ViT 75 %	**65.9**	**21.0**	**30.9**	**26.5**	**10.1**	**173.2**	**1.12×**
	EP-ViT 50 %	**47.9**	**20.7**	**30.5**	**26.2**	**10.0**	**153.9**	**1.26×**
`high_motion`	Dense B2d	89.1	21.3	31.3	26.7	10.6	196.1	1.00×
	EP-ViT 75 %	**67.3**	**21.0**	**30.7**	**26.5**	**10.6**	**176.9**	**1.11×**
	EP-ViT 50 %	**49.4**	**20.7**	**30.4**	**26.4**	**11.2**	**155.7**	**1.26×**

Which tokens are computed. Figure 8.4 shows the tokens of each frame at a 75 % budget. Every P-frame computes the same number of tokens, and the two video classes differ in which ones they are. On `normal`, 11.0 % of all tokens are computed because the codec marks them unmatchable, while 49.0 % are matchable and 15.0 % static tokens computed only to fill the budget; of the 25 % reused, 20.2 % stay in place and 4.8 % are warped. On `high_motion`, 40.1 % are unmatchable, almost no static tokens exist (0.5 %), and 18.8 % are warped. Per-video share of unmatchable tokens also moves widely: on `sav_001799` it reaches the budget on some frames, which then spend all of it on unmatchable tokens. As the budget falls from 90 to 50 %, `normal` keeps every unmatchable token (11 %) and gives up static and matchable ones, whereas

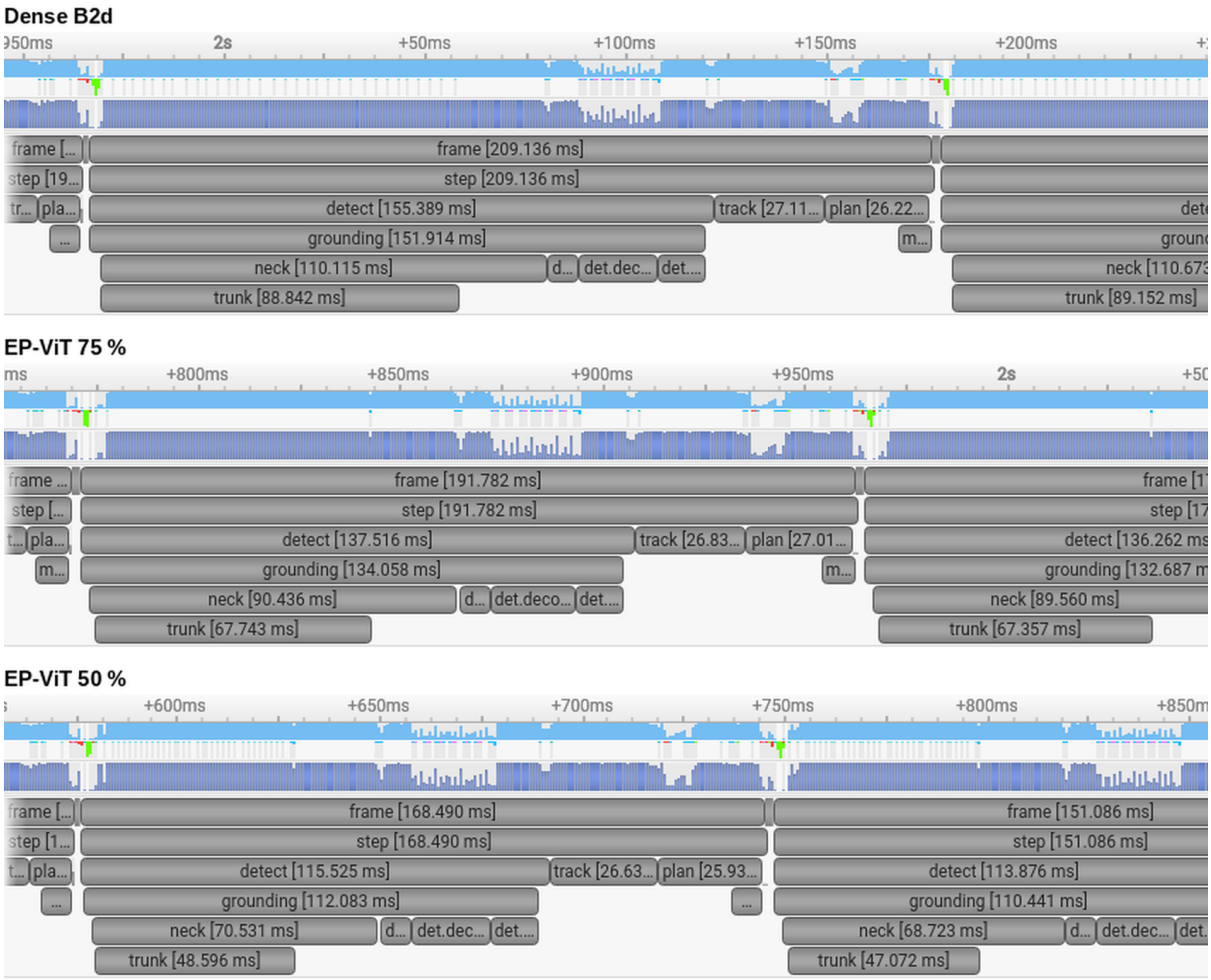


Figure 8.1: Nsight Systems timelines on `normal` (`sav_012075`, "a mic stand"), at one time scale. In each panel, the top rows are the GPU kernels, and the rows below are the NVTX ranges of the frame, step and pipeline stages, where the neck range contains the trunk. The first full frame is the ninth captured frame in each panel, a slower frame than the median in all three methods. Screenshots of the Nsight Systems GUI, with process and memory rows removed.

`high_motion` can no longer afford them all, and its computed unmatchable share falls from 43 to 33 %. I-frames are rare (51 of 7616 frames), and so nearly every frame runs the sparse path.

Accuracy against latency. Table 8.4 gives the trade-off for all settings. A 100 % budget computes the dense function, and its pHOTA differs from dense by rounding only (−0.07 and −0.01 points). It is 3 % slower than dense end to end, because of the cost of EP-ViT's own attention path, which every setting pays. From the 90 % budget on, EP-ViT is faster than dense.

On `normal`, pHOTA falls steadily with the budget, from 55.45 for dense to 54.25 at 75 % and 53.83 at 50 %. The tokens EP-ViT skips there are mostly static or matchable according to the codec, so the loss comes from features that change although their pixels do not, which the codec cannot detect (Section 2.4). On `high_motion`, the scores are not monotonic: C and D score above dense (39.68 and 38.88 against 38.35), and F loses 2.06 points. The output moves much further from dense there, with a union-mask IoU of 0.72 at 75 % against 0.88 on `normal` (the eager runs; two exact dense configurations agree to 0.97 and 0.99, as does the 100 % setting). EP-ViT changes many predictions on this class, and the changes raise and lower the score in roughly equal measure. cgF_1 does not separate the settings: its changes are mostly smaller than

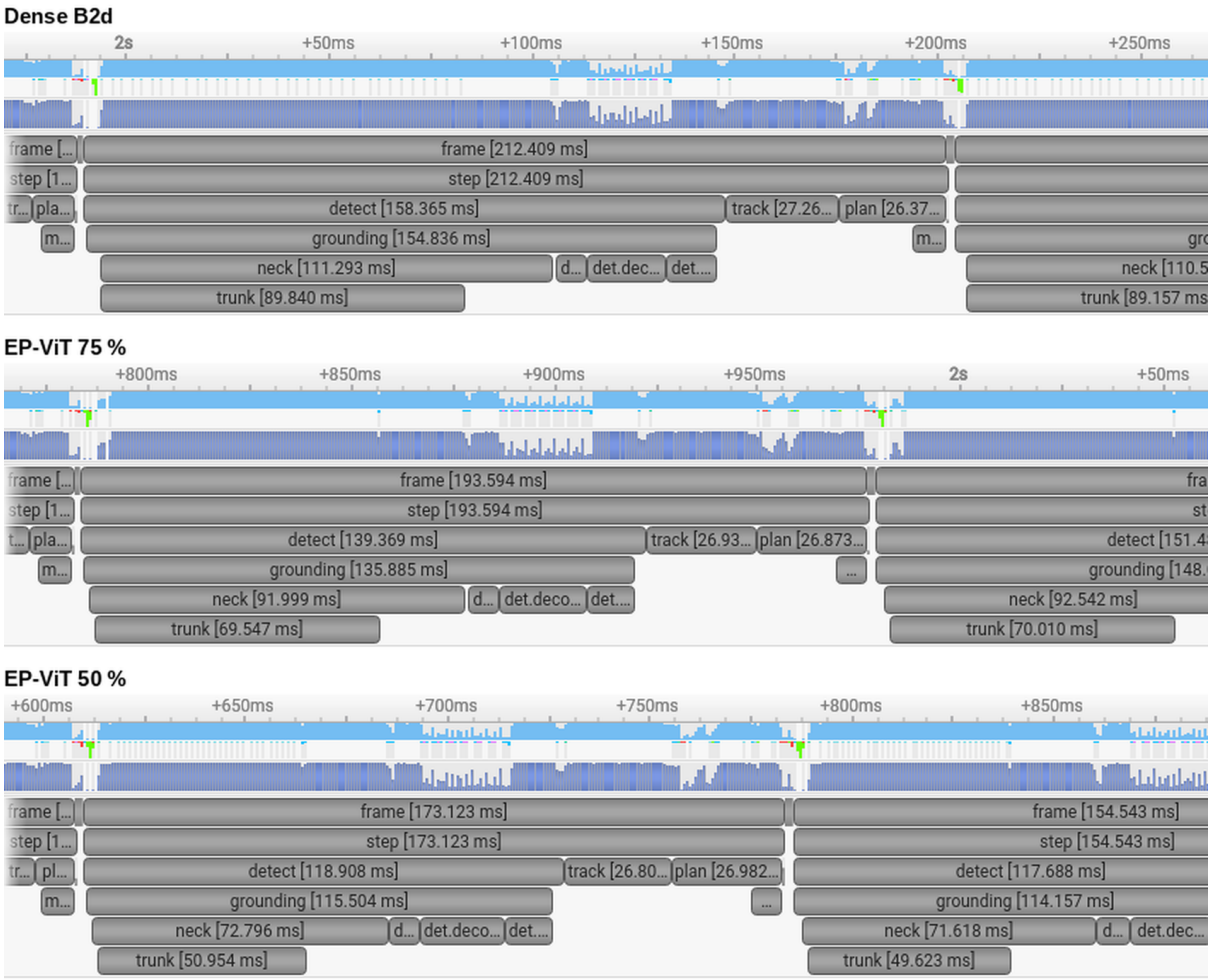


Figure 8.2: The same view as Fig. 8.1 on `high_motion` (`sav_036663`, "black helmet").

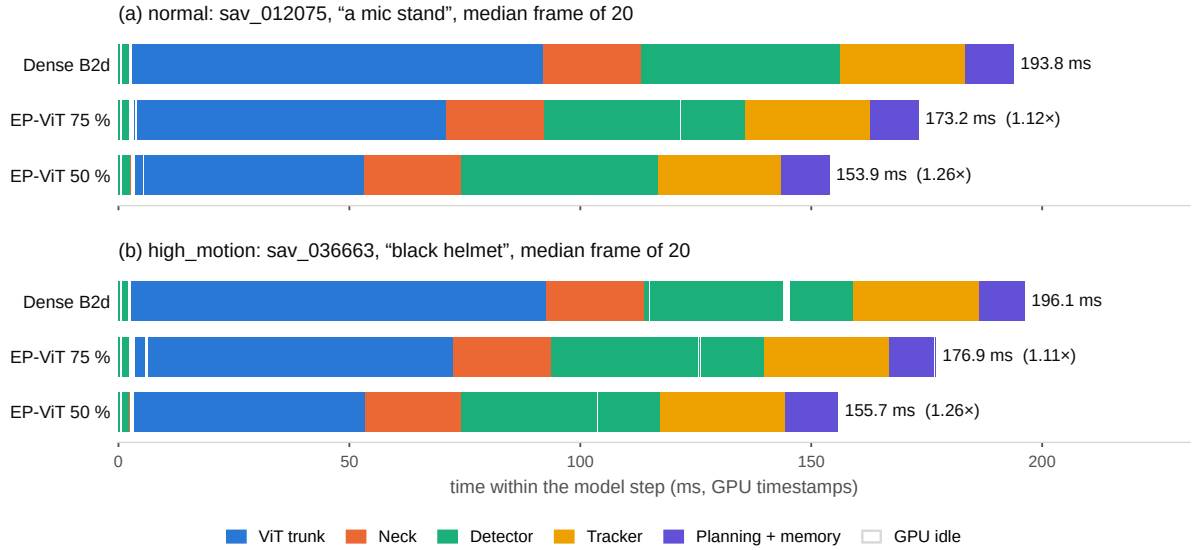


Figure 8.3: GPU timeline of one median frame, drawn from the Nsight Systems traces and coloured by pipeline stage. (a) `normal`: `sav_012075`, "a mic stand". (b) `high_motion`: `sav_036663`, "black helmet". Gaps are GPU idle time. Speedups are against dense B2d of the same pair.



Figure 8.4: Tokens computed and reused by EP-ViT. (a, c) One bar per frame at a 75 % budget for a `normal` and a `high_motion` video; black bars are I-frames, on which all tokens are computed. (b, d) Mean over the P-frames of all ten videos of the class at four budgets. Classes are the codec's; tokens forced back by the update link appear under their codec class.

Table 8.4: Accuracy against end-to-end latency for every EP-ViT setting. α = token budget; Gated = operations that use the budget; GMAC = executed trunk GMAC per frame. pHOTA and cgF$_1$ are from the full evaluation (70 `normal` and 119 `high_motion` pairs). E2E is the controlled measurement of ten videos per class, and S_{e2e} is taken against the dense run of the same round; the dense row gives the mean of its four rounds. Bold: the setting we recommend.

				`normal`				`high_motion`			
Set.	**α**	**Gated**	**GMAC**	**pHOTA**	**cgF$_1$**	**E2E ms**	**S_{e2e}**	**pHOTA**	**cgF$_1$**	**E2E ms**	**S_{e2e}**
Dense	–	–	2699	55.45	15.43	187.0	1.00×	38.35	11.00	188.0	1.00×
A	100 %	all	2686	55.37	14.04	191.9	0.97×	38.34	11.10	192.8	0.97×
B	90 %	all	2434	55.34	16.48	179.7	1.04×	37.21	10.11	181.3	1.04×
C	80 %	all	2180	54.47	16.79	174.5	1.07×	39.68	11.14	176.4	1.06×
D	**75 %**	**all**	**2052**	**54.25**	**14.63**	**163.6**	**1.15×**	**38.88**	**11.11**	**165.8**	**1.13×**
E	60 %	all	1663	53.94	16.98	156.9	1.20×	37.25	9.87	159.2	1.19×
F	50 %	all	1400	53.83	17.00	147.8	1.27×	36.28	8.40	150.1	1.25×
G	75 %	QKV	2557	54.55	14.37	185.8	1.01×	38.64	10.47	187.6	1.00×
H	75 %	MLP	2288	53.41	14.43	173.6	1.08×	38.76	9.71	175.4	1.07×

the 1.7-point difference between two exact dense presets (Section 8.1), the one exception being F on `high_motion`, at −2.6 points.

We recommend setting D. Over all 20 videos it loses 0.61 pHOTA points for a 24 % saving in MACs and a 1.13–1.15× end-to-end gain. E and F add end-to-end speed (up to 1.20× and 1.27×), and cost 1.34 and 1.62 points. We have not computed bootstrap intervals for these runs, so we call none of the pHOTA differences significant; by the Casebook's estimate (Section 7.4) differences below about two points cannot be resolved on these subsets. Gating fewer operations is not worth it. QKV-only gating (G) saves 5 % of the MACs and no time, because the saving is consumed by the gathers and buffer writes it needs. MLP-only gating (H) reaches only 1.07–1.08×, and loses 2.03 points on `normal`.

Overheads outside the trunk's arithmetic. The host-side scheduler takes 0.61–0.73 ms per frame (median) and the plan upload 0.22–0.26 ms; both are included in the encoder time. The reuse state takes 668 MiB for the full method (key and value buffers of all 32 blocks), 972 MiB for QKV-only gating and 324 MiB for MLP-only gating. This raises the median per-pair peak memory by up to 0.8 GiB; on `high_motion` the maximum, set by the tracker on the most crowded pair, stays at 23.1–23.4 GiB. The codec work is not in any timing. Building the priors offline costs 162 ms per frame on our host's CPU (33 ms H.264 encoding, 8 ms decoding with motion-vector export and 121 ms of token mapping in Python), several times the step time it saves. EP-ViT saves time only where motion vectors come with the video for free.

9 GPU Profiling and Bottleneck Analysis

9.1 Dense ViT Profiling

[TODO: Dense ViT Profiling.]

9.2 Sparse Execution Overhead

[TODO: Sparse Execution Overhead.]

9.3 MAC Reduction vs. GPU Speedup

[TODO: MAC Reduction vs. GPU Speedup.]

9.4 Kernel Optimization Opportunities

[TODO: Kernel Optimization Opportunities.]

10 Discussion and Limitations

[TODO: Discussion and Limitations.]

11 Conclusion and Future Work

[TODO: Conclusion and Future Work.]

References

[1] M. Dutson, Y. Li, and M. Gupta, "Eventful transformers: Leveraging temporal redundancy in vision transformers," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 16 911–16 923.

[2] Y. Qiu, Q. Wang, Z. Li, and Q. Liu, "EP-ViT: Leveraging encoding priors to eliminate spatio-temporal redundancy in vision transformer for efficient inference," in *Proceedings of the European Conference on Artificial Intelligence (ECAI)*, ser. Frontiers in Artificial Intelligence and Applications, vol. 413. IOS Press, 2025, tODO: add page numbers.

[3] N. Carion *et al.*, "SAM 3: Segment anything with concepts," in *International Conference on Learning Representations (ICLR)*, 2026, arXiv:2511.16719.

[4] Meta Superintelligence Labs, "SAM 3.1 release notes: Object multiplex," https://github.com/facebookresearch/sam3/blob/main/RELEASE_SAM3p1.md, Mar. 2026, released 27 March 2026.

[5] Y. Li, H. Mao, R. Girshick, and K. He, "Exploring plain vision transformer backbones for object detection," in *European Conference on Computer Vision (ECCV)*, 2022.

[6] Z. Wang, E. P. Simoncelli, and A. C. Bovik, "Multiscale structural similarity for image quality assessment," in *Proceedings of the 37th Asilomar Conference on Signals, Systems and Computers*, 2003, pp. 1398–1402.

[7] G. Farnebäck, "Two-frame motion estimation based on polynomial expansion," in *Image Analysis: 13th Scandinavian Conference (SCIA)*, ser. Lecture Notes in Computer Science, vol. 2749, 2003, pp. 363–370.

[8] T. Wiegand, G. J. Sullivan, G. Bjøntegaard, and A. Luthra, "Overview of the H.264/AVC video coding standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 560–576, 2003.

[9] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the high efficiency video coding (HEVC) standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 22, no. 12, pp. 1649–1668, 2012.

[10] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, "Segment anything," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[11] N. Ravi *et al.*, "SAM 2: Segment anything in images and videos," in *International Conference on Learning Representations (ICLR)*, 2025.

[12] D. Bolya *et al.*, "Perception encoder: The best visual embeddings are not at the output of the network," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv:2504.13181.

[13] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *European Conference on Computer Vision (ECCV)*, 2020.

[14] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021.

[15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[16] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," arXiv preprint arXiv:1607.06450, 2016.

[17] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," arXiv preprint arXiv:1606.08415, 2016.

[18] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," arXiv preprint arXiv:2104.09864, 2021.

[19] G. J. Sullivan and T. Wiegand, "Rate-distortion optimization for video compression," *IEEE Signal Processing Magazine*, vol. 15, no. 6, pp. 74–90, 1998.

[20] B. K. P. Horn and B. G. Schunck, "Determining optical flow," *Artificial Intelligence*, vol. 17, no. 1–3, pp. 185–203, 1981.

[21] C.-Y. Wu, M. Zaheer, H. Hu, R. Manmatha, A. J. Smola, and P. Krähenbühl, "Compressed video action recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 6026–6035.

[22] J. Luiten, A. Ošep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe, "HOTA: A higher order metric for evaluating multi-object tracking," *International Journal of Computer Vision*, vol. 129, pp. 548–578, 2021.

A Hyperparameters and Configuration

[TODO: Hyperparameters and Configuration.]

B Additional Results

[TODO: Additional Results.]

C Profiling Details

[TODO: Profiling Details.]